

 Université Paris Cité	BUT 1 Groupe 110 XU Sarah Groupe 108 XU Célia
--	--

BUT1 – SAE S1.02

Comparaison d’approches algorithmiques

– *Octo-Verso* –

Rapport du projet C

Ce dossier, conçu dans le cadre de notre **projet de développement informatique**, fournit une analyse complète et structurée de notre travail. Il s'ouvre sur une page de garde mentionnant les noms des membres du binôme ainsi que l’objet du document, suivie d’une table des matières facilitant la navigation. Une **introduction** concise présente le projet, avant de laisser place à un **graphe de dépendance** des fichiers sources, offrant une vision synthétique des relations entre les différents composants développés et utilisés. Une section est ensuite consacrée aux **tests unitaires**, mettant en lumière ceux réussis et ceux échoués, offrant ainsi une transparence sur la robustesse de notre code. Le **bilan du projet** aborde les défis rencontrés, nos succès, et les perspectives d'amélioration. Enfin, une annexe structurée regroupe l’intégralité du code source.

TABLE DES MATIÈRES

I - Présentation du Projet	3
a- Rôle fonctionnel du projet	3
b- Entrées et sorties de l'application	4
II – Structuration du code.....	5
a- Graphe de dépendance des fichiers sources	5
b- Rôle fonctionnel des différentes sources	6
III – Éléments de correction.....	7
a- Le code source des tests unitaires pour les structures	7
b- Test des fonctions du jeu	18
IV – Bilan du Projet	21
a- Difficultés rencontrées	21
b- Réussites	22
c- Pistes d'améliorations	22
ANNEXES	23

I - Présentation du Projet

a- Rôle fonctionnel du projet

Le projet consiste à développer un logiciel permettant à deux joueurs de jouer au jeu de lettres **Octo-Verso**. Créé par Laurence Alsac en 2010, **Octo-Verso** utilise un paquet de **88 chevalets**, chacune associée à une **lettre** de l'alphabet en majuscule (excepté K, X, Y, Z et W) et un **rail rotatif** qui peut comporter **8 chevalets** au maximum. Le but du jeu est d'être le premier joueur à se débarrasser de tous ses chevalets, en respectant les règles du jeu. Les joueurs sont désignés par leur numéro d'ordre (1 et 2).

Pour débiter la partie, **12 chevalets** sont tirés au hasard de la pioche et sont remis à chacun des deux joueurs. Les joueurs forment chacun un **mot de 4 lettres** à partir de leurs chevalets et les deux mots sont posés côte à côte sur le rail dans l'ordre alphabétique. Le joueur ayant proposé le **plus petit**¹ mot de 4 lettres commence.

Chaque joueur, à son tour, peut effectuer **l'une des actions suivantes** :

- former un mot** avec au moins deux chevalets du rail en glissant ses chevalets sur une extrémité au choix du rail recto ou du rail verso. L'expulsion d'un nombre de chevalets équivalents à l'autre extrémité est récupérée par le joueur adverse. Si le mot est formé de 8 lettres, le joueur **enlève** ensuite un de ses chevalets au choix qui rejoint la pioche.
- échanger** un de ses chevalets qu'il choisit avec un chevalet de la pioche tiré au hasard.
- signaler** que l'adversaire pouvait former un mot de 8 lettres après que son adverse ait formé un mot de moins de 8 lettres et ensuite il se **défausse** d'un de ses chevalets qui rejoint la pioche.

Le jeu continue jusqu'à ce qu'un joueur n'ait plus de chevalets, signifiant qu'il a remporté la partie, déclenchant la fin de la partie.

L'application doit gérer le **mélange** de la pioche, la **distribution**, la comparaison (ordre alphabétique), et **afficher** la situation du jeu à chaque étape. Les coups doivent être saisis sur une seule ligne de texte, avec une gestion appropriée des erreurs. Le **dictionnaire** de la langue française est utilisé pour vérifier la validité des mots et un même **mot** ne peut être joué qu'une **seule fois** au cours d'une même partie. La partie se termine **lorsqu'un des deux joueurs n'a plus de chevalets**, et le programme se clôture automatiquement.

¹ celui qui vient **avant l'autre en ordre alphabétique**

b- Entrées et sorties de l'application

Ici, le jeu est donc composé de **7 commandes** que l'utilisateur, le joueur peut effectuer après un 1> ou -1> ou un 2> ou -2>. Cette entrée sur l'entrée standard² nécessite plusieurs **préconditions**. Le « **mot** » que le joueur pose est composé : - des lettres à une seul extrémité du rail (entre parenthèses) et les lettres du joueur avant ou après les parenthèses.

Tout d'abord, le **dictionnaire** français, nommé « ods4.txt » doit impérativement se situer dans le répertoire courant à l'application³.

Ci – dessous sont représentés les entrées et sorties de l'application :

COMMANDE	ENTREES	SORTIES
entrer	« MOT »	Rien, mais les 4 lettres du mot que possède le joueur sont posées dans le rail recto.
R : Former mot sur le rail R	« R » + « mot »	Rien, mais si les conditions sont remplies : enlève les lettres chez le joueur et glisse les lettres du mot à l'extrémité du rail R, l'autre extrémité expulsée est donnée à l'adversaire.
V : Former mot sur le rail V	« V » + « mot »	Rien et de même, mais on glisse les lettres du mot à l'extrémité du rail V.
– : échanger	« – » + « lettre »	Rien, mais échange la lettre(=chevalet) du joueur avec une autre provenant de la pioche (file).
r : Signalement_R	« r » + « mot »	Rien, mais signalement du mot composé de 8 lettres que l'adversaire pouvait jouer au coup précédant sur le rail R.
v : Signalement_V	« v » + « mot »	Rien, de même mais sur le rail V.
défausser	« lettre »	Rien, mais enlève cette lettre(=chevalet) du joueur et la pose dans la pioche.
		De plus, affichage de la partie (chevalets des joueurs, R et V) à chaque tour et à la fin de la partie sort du programme.

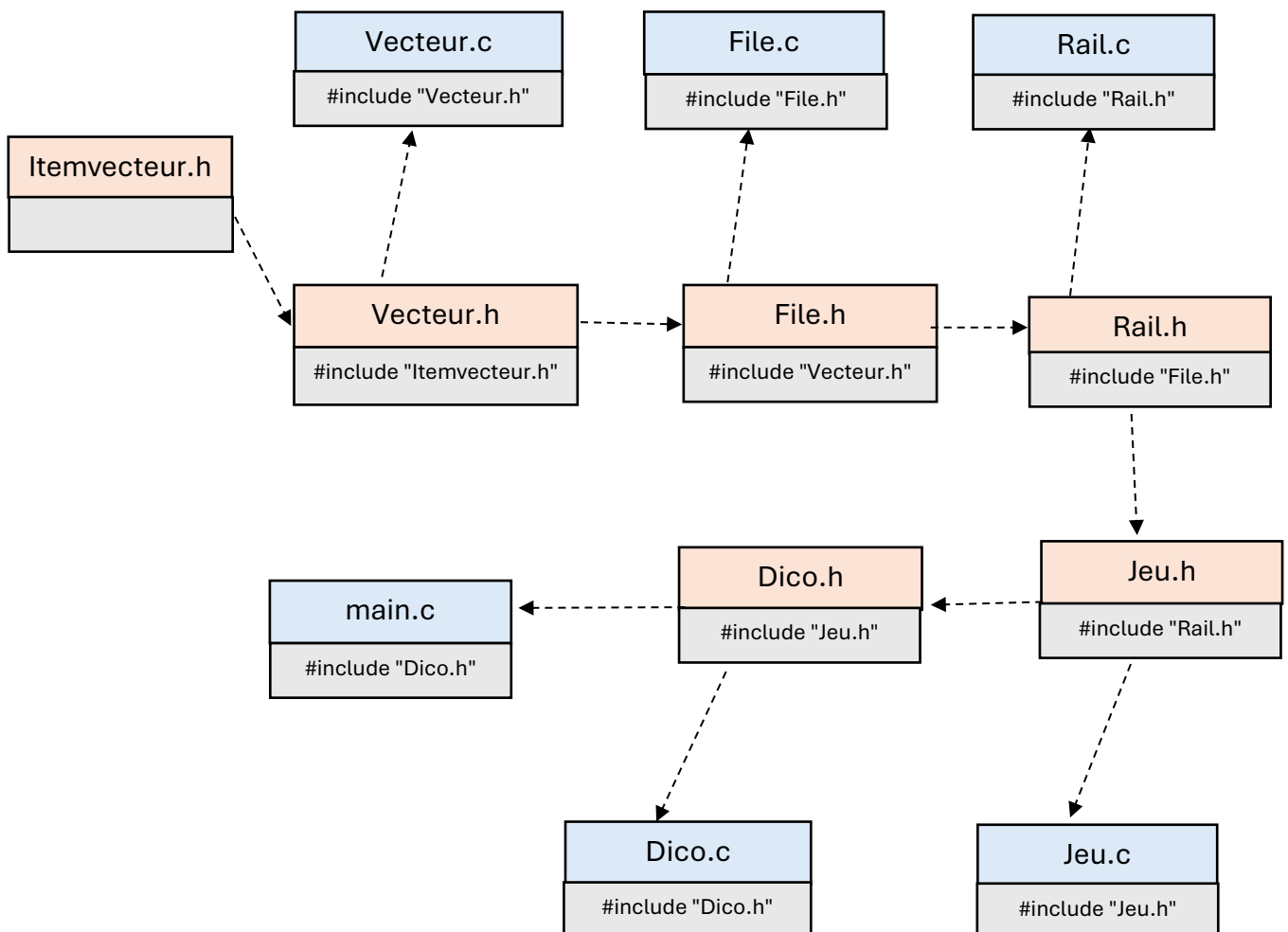
² Ici, le clavier

³ Où il y a l'exécutable, aussi le nom du dictionnaire doit être strictement le même

II – Structuration du code

a- Graphe de dépendance des fichiers sources

Afin, d'efficacement mettre en valeur la structure de notre code, nous avons réalisé un **graphe de dépendance** des fichiers sources. Ici, l'objectif est de montrer nos acquis en termes de **compilation séparée** et de **propreté du code**. Voici le graphe présenté ci-dessous :



b- Rôle fonctionnel des différentes sources

Nom	Fonctions et structures de données associées	Rôle dans le Jeu
main.c		Ici, contient les déclarations et initialisations de toutes les composantes du jeu. Aussi, contient la boucle globale afin de jouer une partie de Octo-Verso.
Dico.h/Dico.c	Structure Dico initDico() ajouterMot() afficheDico() rechercheDico() libererDico() jouerCoup()	La structure Dico est composée d'un tableau alloué dynamiquement permettant d'enregistrer les mots utilisés dans une partie, les mots du dictionnaire et vérifier si un mot peut être joué par le biais du flux d'entrée. Fonctions majeures jouerCoup et rechercheDico présentes.
Jeu.h/Jeu.c	Structure Numero trier() afficherDeckJoueur() rechercheJoueur() supprimerLettre() ajoutMot() copie() ChangeNb() AlternerJoueur() situationActu() traiterLettre() echgChevalet() Init()	Structure Numero permettant d'identifier les joueurs par un numéro qui leur est attribué tout au long du jeu. Fonctions permettant le bon déroulé d'une partie. Aussi, les deux fonctions rechercheJoueur , situationActu , traiterLettre et echgChevalet sont présentes. Puis fonctions subsidiaires permettant leur bon fonctionnement.
Rail.h/Rail.c	DEBUT_MOT = 2, MAX_SAISI = 30, MAX_LETTRE = 10, NB_CARACTERES_STOCKE = 3 ajoutRail() MisEnPlaceRail() afficheRail() copieRail() swap() DansRail() dansParentheses() lireMotSansParentheses() HorsParentheses() editRail() extraireRail() ajoutArr() ajoutAvant() ajustAvant() ajustD() VerifRail() updRail()	Fonctions permettant le bon déroulé d'une partie et surtout la gestion du rail rotatif : rail recto R et rail verso V. Aussi, les fonctions MisEnPlaceRail et VerifRail sont présentes. Puis fonctions subsidiaires permettant le bon fonctionnement du rail.
File.h/File.c	LETTRES = 21, MAX_CHEVALETS=88 Structure File donnée en cours... Rajout : afficherPioche() creerDeck() melange() ajout() distribution() ajoutPioche()	La structure File permet la création creerDeck et la gestion de la pioche et également l'échange et la distribution des 12 chevaux de départ aux joueurs avec ajout .
Vecteur.h/Vecteur.c	MAX_C = 12, MAX_R = 8, MOT_DE_QUATRE = 4, MAX_MOT = 8 Structure Vecteur donnée en cours...	Structure Vecteur ⁴ et fonctions permettant de gérer les chevaux des joueurs et les chevaux sur le rail rotatif.
Itemvecteur.h	Typedef ItemV	Spécification de l'ItemV en char

⁴ Ici, dans mon programme, un **vecteur** représente les chevaux d'un **joueur** et **vecteur** représente les chevaux sur un sens du **rail**.

III – Éléments de correction

a- Le code source des tests unitaires pour les structures

Afin d'assurer la **fiabilité** de notre **code**, nous avons **systematiquement testé** le programme **après** chaque **modification**. Cette phase critique a été essentielle pour **déceler et rectifier** les **failles** dans nos algorithmes. Pour réaliser ces tests, nous avons utilisé de **tests unitaires**, que nous avons utilisés sur nos fonctions au fur et à mesure, voici quelques exemples (**tous les tests passent**) :

```
#include <assert.h>
#include <stdio.h>
#include "file.h"

int main() {
    File f;
    Vecteur v;

    // Test d'initialisation
    initFile(&f, 20);
    assert(estVide(f));
    printf("Test initialisation : OK\n");

    // Test d'entrée et de sortie
    entrer(&f, 'A');
    entrer(&f, 'B');
    entrer(&f, 'C');
    assert(!estVide(f));
    assert(premier(&f) == 'A'); // Premier élément
    printf("Test entrée : OK\n");

    char c1 = sortir(&f); // Sort 'A'
    assert(c1 == 'A');
    assert(premier(&f) == 'B'); // Nouveau premier élément
    printf("Test sortie : OK\n");

    // Test d'initialisation
    initFile(&f, 20);
    assert(estVide(f));
    printf("Test initialisation : OK\n");
    // Test de création d'un deck
    creerDeck(&f);
    assert(f.nbElements > 0); // Vérifie que des lettres ont été ajoutées
    printf("Test création de deck : OK\n");

    // Test de mélange
    File copie = f; // Créer une copie pour comparer avant et après
    melange(&f);
    assert(f.nbElements == copie.nbElements); // Même nombre d'éléments
    printf("Test mélange : OK\n");

    // Test d'ajout au vecteur
    initVecteur(&v, 12); // Initialise à une capacité de 12.
    ajout(&v, &f); // Ajoute 12 lettres (ou moins selon disponibilité)
```

```

    assert(taille(&v) == 12); // Vérifie que le vecteur est rempli à sa
    capacité
    printf("Test ajout au vecteur : OK\n");

    // Test de distribution
    distribution(&v, &f);
    assert(taille(&v) == 13); // Une lettre de plus distribuée
    assert(f.nbElements == copie.nbElements - 1); // Une lettre en moins
    dans la file
    printf("Test distribution : OK\n");

    // Test d'ajout manuel dans la pioche
    ajoutPioche(&f, 'Z');
    assert(f.elements[f.nbElements - 1] == 'Z'); // Dernier élément est 'Z'
    printf("Test ajout pioche : OK\n");

    // Test destruction
    detruireFile(&f);
    detruireVecteur(&v);
    printf("Test destruction : OK\n");

    printf("\nTous les tests unitaires sont passés avec succès.\n");
    return 0;
}

```

Voici le résultat de la compilation ainsi les tests du fonctionnement de la file : surtout le mélange de la pioche, la distribution et l'échange passent.

```

Test initialisation : OK
Test entrée : OK
Test sortie : OK
Test initialisation : OK
Test création de deck : OK
Test mélange : OK
Test ajout au vecteur : OK
Test distribution : OK
Test ajout pioche : OK
Test destruction : OK

Tous les tests unitaires sont passés avec succès.

C:\Users\celia\Downloads\Brlllloonn\x64\Debug\nule.exe (processus 24028) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .|

```

Vecteur qui est la structure la plus adaptée pour gérer les chevaux du rail rotatif et des joueurs.

```

#include <assert.h>
#include <stdio.h>
#include "vecteur.h"

int main() {
    Vecteur v;

    // Test de l'initialisation
    assert(initVecteur(&v, 5));
    assert(taille(&v) == 0);

    // Test de l'ajout d'éléments
    assert(ajouter(&v, 'A'));
    assert(ajouter(&v, 'B'));
}

```



```

    assert(ajouter(&v, 'C'));
    assert(taille(&v) == 3);
    assert(obtenir(&v, 0) == 'A');
    assert(obtenir(&v, 1) == 'B');
    assert(obtenir(&v, 2) == 'C');

    // Test de la modification
    modifier(&v, 1, 'D');
    assert(obtenir(&v, 1) == 'D');

    // Test de suppression
    supprimer(&v, 1); // Supprime 'D'
    assert(taille(&v) == 2);
    assert(obtenir(&v, 0) == 'A');
    assert(obtenir(&v, 1) == 'C');

    // Test de redimensionnement
    assert(retailler(&v, 10));
    assert(v.capacite == 10);

    // Test de dépassement de capacité et agrandissement
    for (int i = 0; i < 10; ++i) {
        assert(ajouter(&v, 'X' + i));
    }
    assert(taille(&v) == 12); // 2 initialement + 10 ajoutés
    assert(v.capacite >= 12); // Vérifie la capacité agrandie

    // Test de destruction
    detruireVecteur(&v);

    printf("Tous les tests unitaires sont passés avec succès.\n");
    return 0;
}

```

Ainsi les tests pour la gestion d'un vecteur : l'ajout ou la suppression d'un chevalets passent.

```

Tous les tests unitaires sont passés avec succès.
C:\Users\celia\Downloads\Brilllonnn\x64\Debug\nule.exe (processus 26860) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .

```

Dico pour l'enregistrement des mots du dictionnaire et des mots utilisés dans une partie.

```

#include <stdio.h>
#include <assert.h>
#include <string.h>
#include "dico.h"

void test_initDico() { //Initialisation du dictionnaire
    Dico d;
    initDico(&d);
    assert(d.qte == 0);
    assert(d.mots == NULL);
    printf("test_initDico passe.\n");
}

```

```

void test_ajouterMot() {           //Ajout mot(s), s'il est correctement
ajoutés.
    Dico d;
    initDico(&d);

    ajouterMot(&d, "chat");    //L'ajout
    assert(d.qte == 1);
    assert(strcmp(d.mots[0], "chat") == 0);    //Vérification de l'ajout

    ajouterMot(&d, "chien");
    assert(d.qte == 2);
    assert(strcmp(d.mots[1], "chien") == 0);

    printf("test_ajouterMot passe.\n");
    libererDico(&d);
}

void test_rechercheDico() {    //Fonction recherche dans dictionnaire si le
mot existe.
    Dico d;
    initDico(&d);
    ajouterMot(&d, "pomme");    //Ajout de(s) dans le dictionnaire
    ajouterMot(&d, "banane");

    assert(rechercheDico(&d, "pomme") == 1);    //Utilise la fonction
rechercheDico pour vérifier
    assert(rechercheDico(&d, "banane") == 1);
    assert(rechercheDico(&d, "cerise") == 0);

    printf("test_rechercheDico passe.\n");
    libererDico(&d);
}

void test_erreurs() {    //Test les erreurs lors de l'ajout des mots
    Dico d;
    initDico(&d);

    ajouterMot(&d, NULL);    // Ne doit pas être ajouté
    assert(d.qte == 0);

    ajouterMot(&d, "");    // Ne doit pas être ajouté
    assert(d.qte == 0);

    assert(rechercheDico(&d, NULL) == 0);    //Vérification s'il y a bien des
erreurs
    assert(rechercheDico(&d, "") == 0);

    printf("test_erreurs passe.\n");
    libererDico(&d);
}

void test_libererDico() {    //Libération du dictionnaire
    Dico d;
    initDico(&d);
    ajouterMot(&d, "soleil");    //ajout de(s) mot(s)
    ajouterMot(&d, "lune");

    libererDico(&d);                //désallouement
    assert(d.qte == 0);            //Vérification qu'il n'y a bien rien
    assert(d.mots == NULL);
}

```

```

    printf("test_libererDico passe.\n");
}

int main() {
    test_initDico();
    test_ajouterMot();
    test_rechercheDico();
    test_erreurs();
    test_libererDico();

    printf("\nTous les tests sont passes avec succes !\n");
    return 0;
}

```

```

test_initDico passe.
test_ajouterMot passe.
test_rechercheDico passe.
Mot invalide.
Mot invalide.
erreur (rechercheDico)
erreur (rechercheDico)
test_erreurs passe.
test_libererDico passe.

Tous les tests sont passes avec succes !

C:\Users\celia\Downloads\Brlllloonn\64\Debug\nule.exe (processus 25416) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .

```

Ainsi, l'enregistrement des mots et la recherche de mots dans l'enregistrement passent.

Voici les tests sur les fonctions principales du jeu :

```

#include <stdio.h>
#include <assert.h>
#include <string.h>
#include <stdlib.h>
#include "joueur.h"
#include "vecteur.h"

void test_trier() {
    char chevalet[] = { 'd', 'a', 'c', 'b', 'e', '\0' }; //Ajout lettres dans le
    tableau
    trier(chevalet, 5); //Trie un tableau (alphabétique)
    assert(strcmp(chevalet, "abcde") == 0); //Si le tableau est correctement
    trier
    printf("test_trier PASSE\n\n");
}

void test_rechercheJoueur() {
    Vecteur v;
    initVecteur(&v, 10); //Initialise avec une capacité de 10
    const char* mot = "abcdef"; //Chaîne caractère contenant abcdef
    for (int i = 0; i < strlen(mot); ++i) { //Parcourt lettre par lettre
        if (!ajouter(&v, mot[i])) {
            printf("mal ajouté");
            break;
        }
    }
    //1 pour vrai, si les lettres sont bien présentes
    //0 si les lettres ne sont pas toutes présentes
    assert(rechercheJoueur(&v, "abc") == 1);
    assert(rechercheJoueur(&v, "fed") == 1);
    assert(rechercheJoueur(&v, "face") == 1);
}

```

```

    assert(rechercheJoueur(&v, "zoo") == 0);
    assert(rechercheJoueur(&v, "aaaa") == 0);
    printf("Affichage d'erreur si Joueur n'a rien:\n");
    assert(rechercheJoueur(&v, "") == 0);
    assert(rechercheJoueur(&v, NULL) == 0);
    printf("test_rechercheJoueur PASSE\n\n");
    detruireVecteur(&v);    //désallouement du vecteur
}

void test_afficherDeckJoueur() {
    Vecteur v;
    initVecteur(&v, 10);
    const char* mot = "xhy";
    for (int i = 0; i < strlen(mot); ++i) {
        if (!ajouter(&v, mot[i])) {
            printf("mal ajouté");
            break;
        }
    }
    printf("Deck attendu: xhy\nDeck affiche : "); //Si le mot ajouté correspond
à celle afficher
    afficherDeckJoueur(v);
    printf("test_afficherDeckJoueur PASSE\n\n");
    detruireVecteur(&v);
}

int main() {
    test_trier();
    test_rechercheJoueur();
    test_afficherDeckJoueur();
    printf("Tous les tests sont PASSES !!!!!\n");
    return 0;
}

```

```

test_trier PASSE

Affichage d'erreur si Joueur n'a rien:
Mot invalide.
Mot invalide.
test_rechercheJoueur PASSE

Deck attendu: xhy
Deck affiche : xhy
test_afficherDeckJoueur PASSE

Tous les tests sont PASSES !!!!!

C:\Users\celia\Downloads\Brllllonnn\x64\Debug\nule.exe (processus 30436) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .

```

Ainsi les tests de vérification des lettres que possèdent le joueur et leur affichage passent.

```

void test_trier_et_afficherDeckJoueur() {
    Vecteur deck_j;
    initVecteur(&deck_j, 5);
    // Ajout des lettres
    ajouter(&deck_j, 'L');
    ajouter(&deck_j, 'E');
    ajouter(&deck_j, 'S');
    //Vérification des éléments, s'ils sont bien ajoutés
    assert(deck_j.nbElements == 3);
    assert(deck_j.elements[0] == 'L');
    assert(deck_j.elements[1] == 'E');
    assert(deck_j.elements[2] == 'S');
}

```

```

printf("Deck avant tri: ");
afficherDeckJoueur(deck_j);

trier(deck_j.elements, deck_j.nbElements);
char resultatAttendu[] = { 'E', 'L', 'S' };
assert(memcmp(deck_j.elements, resultatAttendu, 3) == 0);

printf("Deck apres tri: ");
afficherDeckJoueur(deck_j);
destruireVecteur(&deck_j);
printf("Tous les tests de tri et d'affichage sont passes");
}

```

```

int main() {
    test_trier_et_afficherDeckJoueur();
}

```

```

Deck avant tri: LES
Deck apres tri: ELS
Tous les tests de tri et d'affichage sont passes
C:\Users\celia\Downloads\Brllllonnn\x64\Debug\nule.exe (processus 20684) s'est termin  avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fen tre. . .

```

```

void test_distribution() {
    File pioche;
    Vecteur joueur;

    initFile(&pioche, 5); //Initialisation de la pioche
    assert(estVide(pioche)); //La file est vide
    initVecteur(&joueur, 3);
    //Remplit la pioche
    entrer(&pioche, 'A');
    entrer(&pioche, 'B');
    entrer(&pioche, 'C');
    entrer(&pioche, 'D');
    entrer(&pioche, 'E');
    assert(!estVide(pioche)); //La file ne doit pas  tre vide
    assert(premier(&pioche) == 'A'); //Correspond   la premi re lettre de
la pioche

    printf("Pioche avant: ");
    afficherPioche(pioche);
    distribution(&joueur, &pioche); //Distribution d'une lettre aux joueurs
    printf("Chevalet du joueur:");
    afficherDeckJoueur(joueur);
    printf("Pioche restante: ");
    afficherPioche(pioche);

    destruireVecteur(&joueur);
    destruireFile(&pioche);
}

```

```

int main() {
    test_distribution();
}

```

```

Pioche avant: ABCDE
Chevalet du joueur:A
Pioche restante: BCDE

C:\Users\celia\Downloads\Brllllonnn\x64\Debug\nule.exe (processus 25820) s'est termin  avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fen tre. . .

```

```

void test_saisie_valide() {
    Vecteur joueur;

    initVecteur(&joueur, 5);
    ajouter(&joueur, 'A');
    ajouter(&joueur, 'B');
    ajouter(&joueur, 'C');
    ajouter(&joueur, 'D');
    ajouter(&joueur, 'E');

    const char* saisie = "EB"; //Vérification lettre est bien présente
    assert(rechercheJoueur(&joueur, saisie));
    const char* saisie_invalide = "ZEA"; //Vérification lettre non
présente
    assert(!rechercheJoueur(&joueur, saisie_invalide));
    const char* saisie_vide = ""; //Test si vide, affichage erreur
    assert(!rechercheJoueur(&joueur, saisie_vide));

    //Vérification longueur du saisie et si les lettres sont dans les
chevalets du joueur.
    assert(strlen(saisie) == 2); //Saisie égale à deux lettres
    assert(!strlen(saisie) == NULL); //N'est pas nulle
    assert(strlen(saisie_invalide) != 2); //Pas égale à deux lettres
    assert(strlen(saisie_vide) != 2); //Pas égale à deux lettres

    printf("test_rechercheJoueur marche");
    detruireVecteur(&joueur);
}

int main() {
    test_saisie_valide();
}

```

```

Mot invalide.
test_rechercheJoueur marche
C:\Users\celia\Downloads\Brllllonnn\x64\Debug\nule.exe (processus 31928) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .

```

```

void test_verif_dans_dico() {
    Dico dictionnaire;
    initDico(&dictionnaire);
    const char* mot1 = "GOOD";
    const char* mot2 = "FAIL";
    ajouterMot(&dictionnaire, mot1);
    ajouterMot(&dictionnaire, mot2);
    afficheDico(&dictionnaire); //Si dico contient bien les deux mots

    const char* saisie1 = "GOOD";
    const char* saisie2 = "GOD";
    const char* saisie3 = "FAILE";
    const char* vide = "";
    //Vérification s'ils appartiennent à dictionnaire
    assert(rechercheDico(&dictionnaire, saisie1));
    assert(!rechercheDico(&dictionnaire, saisie2));
    assert(!rechercheDico(&dictionnaire, saisie3));
    assert(!rechercheDico(&dictionnaire, vide)); //Message d'erreur,
lorsque saisie est vide

    libererDico(&dictionnaire);
    printf("test_rechercheDico marche");
}

```

```

}

int main() {
    test_verif_dans_dico();
}

```

```

1: GOOD
2: FAIL
erreur (rechercheDico)
test_rechercheDico marche
C:\Users\celia\Downloads\Brllllonnn\x64\Debug\nule.exe (processus 11228) s'est termi

```

```

void test_tri_plus_petit() {
    Vecteur saisie1;
    Vecteur saisie2;
    Vecteur Rail;
    initVecteur(&saisie1, 5);
    initVecteur(&saisie2, 5);
    initVecteur(&Rail, 8);
    const ItemV* mot1 = "ELLE";
    const ItemV* mot2 = "DANS";
    ajoutMot(&saisie1, mot1);
    assert(saisie1.nbElements == 4);
    assert(saisie1.elements[0] == 'E');
    assert(saisie1.elements[1] == 'L');
    assert(saisie1.elements[2] != 'E'); // Non présente
    assert(saisie1.elements[3] != 'L'); // Non présente
    ajoutMot(&saisie2, mot2);
    //ajoutRail(&Rail, &saisie1, &saisie2);
    printf("\nRail trie directement: ");
    MiseEnPlaceRail(&Rail, &saisie1, &saisie2);
    afficheRail(&Rail);

    printf("\ntest_MiseEnPlaceRail marche");
    printf("\ntest_afficheRail marche");
    printf("\ntest_ajoutRail marche");
    detruireVecteur(&Rail);
    detruireVecteur(&saisie1);
    detruireVecteur(&saisie2);
}

int main() {
    test_tri_plus_petit();
}

```

```

Rail trie directement: DANSELLE

test_MiseEnPlaceRail marche
test_afficheRail marche
test_ajoutRail marche
C:\Users\celia\Downloads\Brllllonnn\x64\Debug\nule.exe (processus 34812) s'est terminé a
Appuyez sur une touche pour fermer cette fenêtre. . .|

```

```

#define MAX_R 8
#define MAX_M 8
#define DEBUT 2

void test_verif_coup_jouable() {
    Vecteur Rail;
    initVecteur(&Rail, MAX_R); //Initialisation de Rail a une
    capacité de 4 chevaux

```

```

const ItemV* motRail = "BLEUCIEL"; //Rail contient "BLEUCIEL"
const ItemV* saisi = "R FA(BLE)"; //Mot formée
const ItemV* saisi2 = "R (EL)LE"; //Un autre mot formée
//extraire les saisis pour vérifier
ItemV mot_lu[MAX_M];
ItemV dans_P[MAX_M];
ItemV hors_P[MAX_M];
ajoutMot(&Rail, motRail); //Ajout "BLEUCIEL" dans le rail
assert(Rail.nbElements == 8); //Vérification de Rail s'ils contient
bien 8 lettres et commence par bleu
assert(Rail.elements[0] == 'B');
assert(Rail.elements[1] == 'L');
assert(Rail.elements[2] == 'E');
assert(Rail.elements[3] == 'U');

lireMotSansParentheses(saisi, mot_lu, DEBUT); //Extraire le mot sans
les parenthèses
assert(strlen(mot_lu) == 5); // Vérification du mot et nombres de
lettres
assert(mot_lu[0] == 'F');
assert(mot_lu[1] == 'A');
assert(mot_lu[2] == 'B');
assert(mot_lu[3] == 'L');
assert(mot_lu[4] == 'E');
dansParentheses(saisi, dans_P); // Extraire les lettres dans la
parenthèses
assert(strlen(dans_P) == 3); // Vérification du mot et nombres de
lettres
assert(dans_P[0] == 'B');
assert(dans_P[1] == 'L');
assert(dans_P[2] == 'E');
assert(DansRail(&Rail, dans_P)); //S'il appartient à RAIL
printf("test_DansRail marche\n");

Vecteur joueur;
initVecteur(&joueur, 4); // Initailisation d'un joueur
ajouter(&joueur, 'A'); // On lui ajoute des lettres
ajouter(&joueur, 'F');
ajouter(&joueur, 'E');
ajouter(&joueur, 'L');
Dico d;
initDico(&d); // Initialise dictionnaire
ajouterMot(&d, "FABLE"); // On lui ajoute des mots
assert(d.qte == 1);
assert(strcmp(d.mots[0], "FABLE") == 0); //Vérification de l'ajout
ajouterMot(&d, "ELLE");
assert(d.qte == 2);
assert(strcmp(d.mots[1], "ELLE") == 0);

HorsParentheses(saisi, hors_P, DEBUT); // Extraire les lettres hors
parenthèses lors de la saisie
assert(rechercheJoueur(&joueur, hors_P)); // Vérification qu'ils
correspondent bien aux chevalets que le joueur a dans ses mains

assert(jouerCoup(&d, &joueur, saisi2, &Rail)); // Vérification avec la
2ème saisie, s'il peut jouer le même coup avec un autre mot
printf("test_jouerCoup marche\n"); // Mot trouvée dans le dictionnaire
et lettre sont dans joueur

```



```

Vecteur adv;
initVecteur(&adv, 4); // Initialisation du vecteur
editRail(&Rail, saisi, &adv); //Saisi: "FABLE"
assert(Rail.nbElements == 8); // Vérification du nombres et les
éléments dans Rail s'ils ont bien été modifiées
assert(Rail.elements[0] == 'F');
assert(adv.nbElements == 2); // Vérification que l'adversaire a
bien reçu les lettres des extrémités du rail "BLEUCIEL" -> "EL"
assert(adv.elements[0] == 'E');
assert(adv.elements[1] == 'L');
printf("test_editRail marche\n"); //Comme la fonction editRail
fonctionne, les fonctions dans cette fonction fonctionnent.
printf("test_extraireRail marche\n"); //Fonction dans editRail.
printf("test_ajoutAvant_ajoutArr_ajustD_ajustAvant marchent\n");
//Fonction dans editRail.

destruireVecteur(&Rail);
destruireVecteur(&joueur);
libererDico(&d);
printf("test_lireMotSansP_DansP_HorsP marchent\n");
}
int main() {
    test_verif_coup_jouable();
}

```

```

test_DansRail marche
test_jouerCoup marche
test_editRail marche
test_extraireRail marche
test_ajoutAvant_ajoutArr_ajustD_ajustAvant marchent
test_lireMotSansP_DansP_HorsP marchent

```

```

C:\Users\celia\Downloads\Brlllloonn\64\Debug\nule.exe (processus 25628) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .

```

```

void test_cas_echgC() {
    File p; //Pioche
    initFile(&p, 5); //Initialisation de la pioche
    assert(estVide(p));
    entrer(&p, 'A');
    entrer(&p, 'B');
    entrer(&p, 'C');
    entrer(&p, 'D');
    entrer(&p, 'E');
    assert(!estVide(p));
    assert(premier(&p) == 'A');

    Vecteur j; //Joueur
    initVecteur(&j, 4); // Initailisation d'un joueur
    ajouter(&j, 'A'); // On lui ajoute des lettres
    ajouter(&j, 'F');
    ajouter(&j, 'E');
    ajouter(&j, 'L');
    assert(j.nbElements == 4); //Vérification nombres de lettres que joueur
a
    const ItemV* lettre = "E"; //lettre à échanger avec la pioche
    echgChevalet(&p, &j, lettre);
    for (int i = 0; i < j.nbElements; ++i) {
        assert(j.elements[i] != 'E'); //Vérification qu'il n'y a bien plus
le 'E'
    }
    assert(j.nbElements == 4); //Reçoit une lettre de la pioche en échange
    printf("test_supprimerLettre_et_distribution marchent\n");
}

```

}

```
int main() {
    test_cas_echgC();
}
```

```
test_supprimerLettre_et_distribution marchent
```

```
C:\Users\celia\Downloads\Brllllonnn\x64\Debug\nule.exe (processus 37768) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .
```

b- Test des fonctions du jeu

Voici un exemple d'une partie dans sa totalité :

```
1 : EEEJLLMORSSS
2 : AEEGNPPRRSTU
```

```
1> SEL
1> sors
1> SORS
2>
2> TUNES
2> POUR
2> PURE
```

```
1 : EEEJLLMS
2 : AEGNPRST
R : PURESORS
V : SROSERUP
```

```
2>
```

Mise en place du jeu

Distribution des 12 chevalets pour chaque joueur.

Les joueurs doivent former un mot de 4 lettres avec les chevalets distribués.

Affichage du courant du jeu et des chevalets de chaque joueur. Mise à jour du courant en fonction du joueur qui a créé le mot le plus petit (alphabétique).



Formation du mot par rapport au RAIL RECTO/VERSO

Formation de mot possible qu'aux extrémités des rails Recto R et Verso V.

```
1 : AAEIM
2 : EIINTTUU
R : OTERMENT
V : TNEMRETO
```

```
2> V (O)U
2> V (O)
2> V NUI(T)
2> R (T)UE
2> V N(ET)
2> V (N)UIT
2> V (T)ETE
2> V (T)OUT
2> V TOUT
2> V (TO)UT
```

```
1 : AAEIMNT
2 : EIINTU
R : TUOTERME
V : EMRETOUT
```

```
1>
```



1> - E

1 : AEIILOPRRTU

2 : AEIR

R : TERACENE

V : ENECARET

2> v (RET)OUPAI

2> |

Signalement 8 lettres qu'aurait pu former l'adversaire

Dans ce cas, impossible. Joueur 2 doit ressaisir.



1> r (TE)RMINEE

1> - I

1 : AEMNT

2 : EEIILMNRT

R : EIAPUOTE

V : ETOUPAIE

2> R (TE)RME

1 : AAEEIMNT

2 : EIILNT

R : PUOTERME

V : EMRETOUP

1> r (TE)RMINEE

1> r (TE)RMINEE

1 : AAEEIMNT

2 : EIILNT

R : PUOTERME

V : EMRETOUP

-1>

-1> d

-1> EEE

-1> E

1 : AAEEIMNT

2 : EIILNT

R : PUOTERME

V : EMRETOUP

1> |

Rail Recto R

Signalement impossible lorsque l'adversaire échange l'un de ses chevaux.

Joueur 2 forme un mot.

Joueur 1 signale qu'il avait pu faire un mot de 8 lettres. Signalement validé. Il peut retirer l'un de ses chevaux et mettre dans la pioche.



Rail Verso V

Pareil.

Signalement impossible
lorsque l'adversaire échange
l'un de ses chevaux

Joueur 1 forme un mot.

Joueur 2 signale. Signalement
validé. Il peut retirer l'un de
ses chevaux et mettre dans
la pioche.

```
1 : AEIILOPRRRTU
2 : AEIO
R : TERACENE
V : ENECARET
```

```
1> v (RET)OUPAI
1> V (RET)OUPAI
```

```
1 : EILRRRT
2 : AEIOENEC A
R : IAPUOTER
V : RETOUPAI
```

```
-1>
```

```
-1> s
```

```
-1> RR
```

```
-1> R
```

```
1 : EILRRT
2 : AACEEEINO
R : IAPUOTER
V : RETOUPAI
```

```
2>
```

```
1> V (PAI)E
```

```
1 : EMNNT
2 : EEIMMNNRS
R : EIAPUOTE
V : ETOUPAIE
```

```
2> v (PAI)EMENT
```

```
1 : EMNNT
2 : EEIMMNNRS
R : EIAPUOTE
V : ETOUPAIE
```

```
-2>
```

```
-2> d
```

```
-2> EE
```

```
-2> E
```

```
1 : EMNNT
2 : EIMMNNRS
R : EIAPUOTE
V : ETOUPAIE
```

```
2>
```



```
1 : EV
2 : ABEEHILNNOQRS
R : ELLEPACE
V : ECAPELLE
```

```
1> V (LE)VE
```

C:\Users\celia\Downloads\BON (1)\BON\x64\Debug\c.exe (processus 20888) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .

Fin du jeu

L'affichage courant n'est pas affiché.

IV – Bilan du Projet

a- Difficultés rencontrées

Dans le cadre de ce projet, nous avons été confrontés à plusieurs **défis**. En effet, gérer de manière robuste et élégante les **erreurs** potentielles introduites par les utilisateurs lors de la **saisie des commandes** a demandé une **attention** particulière. La nécessité d'anticiper et de traiter ces erreurs a ajouté une couche de **complexité** au développement, mais elle a également renforcé notre compréhension des aspects liés à la fiabilité de notre programme.

En outre, l'un des principaux défis aurait été de sélectionner des **structures de données** appropriées et conformes aux exigences du sujet. Par exemple, la structure de **file** est utilisée pour la pioche afin d'éviter de tirer ce qui vient d'être placé dans la pioche. De plus, la structure **vecteur** est employée pour représenter un joueur avec ses chevaux et un vecteur peut également représenter un sens du rail rotatif pour gérer efficacement les ajouts et suppressions. Il a été complexe et fastidieux de mettre en place le système d'attribution des tours aux deux joueurs.

D'un point de vue **algorithmique**, seul le cas pour **défausser** un de ses chevaux a posé problème, car il était implémenté par une boucle while lui-même dans une boucle while avant la dernière version de notre code ce qui affichait -1>-1> si le joueur n'avait pas les chevaux nécessaires pour défausser une lettre. Nous avons donc modifié en conséquence en changeant en do while et en agrandissant la saisie du joueur pour que si l'utilisateur entre plusieurs lettres cela soit bien stocké et compté faux en nous affichant qu'une seule fois par ligne -1> selon une saisie incorrecte. En effet, de nombreuses vérifications ont dû être effectuées, d'où la création de **fonctions subsidiaires** essentielles et nombreuses.

Enfin, les dernières **difficultés** rencontrées furent celle concernant le bon **déroulé** d'une partie en respectant les exigences syntaxiques du sujet. En effet, la mise en place de la **double boucle** while dans le main fut source de nombreuses discussions et débats, car il fallait bien poser les break pour sortir au bon endroit.

b- Réussites

Malgré les défis rencontrés, notre projet a abouti à **des réussites significatives**. La conception réfléchie de nos **structures de données**, l'implémentation des **fonctions clés** du jeu Octo-Verso, et la réalisation de **tests unitaires** approfondis ont contribué à la solidité de notre code. La **collaboration** étroite entre les membres du groupe, l'utilisation judicieuse d'outils collaboratifs tels que **GitHub, Discord** et **Outlook**, et les **retours** constructifs de notre professeur et de camarades ont également constitué des éléments clé de notre réussite. Ainsi, nous avons répondu entièrement aux exigences explicites et implicites du sujet.

c- Pistes d'améliorations

En envisageant des **améliorations potentielles**, nous pourrions explorer des moyens d'**optimiser** davantage les performances de certaines fonctions, notamment celles liées à la manipulation des **chevalets des joueurs** et à la **gestion du rail rotatif**. Si le jeu **évolue**, nous pourrions réaliser une **structure Partie** qui permettrait d'enregistrer les mots utilisés lors d'une partie, enregistrer les joueurs si ce jeu peut permettre plus de deux joueurs et enregistrer le gagnant. Ainsi, plusieurs parties peuvent être comparées pour annoncer le joueur le plus doué du jeu. De plus, une **amélioration de l'interface utilisateur** pour une meilleure expérience de jeu et une prise en charge plus avancée des **erreurs de saisie** pourraient être envisagées comme afficher quels types d'erreur. On pourrait également réaliser les **commandes bonus** :

- La **commande H** permet aux joueurs en perdition d'obtenir de l'aide : tous les meilleurs coups possibles sont affichés s'il saisit H. - Lorsqu'un **joueur** ne peut former **aucun mot de quatre lettres** (pour débiter une partie) et c'est à lui de jouer, le programme se termine sans autre explication : la partie est annulée. Enfin, une extension du programme avec des **fonctionnalités additionnelles** du jeu Recto-Verso pourrait constituer une perspective intéressante pour enrichir notre projet.

En somme, malgré les défis rencontrés, le projet a été une expérience enrichissante, permettant d'appliquer et de consolider nos connaissances en développement informatique. La réussite du projet repose sur la combinaison d'une conception réfléchie, d'une collaboration efficace et d'une approche rigoureuse pour surmonter les défis rencontrés et répondre de manière complète aux exigences du sujet.

ANNEXES

Table des matières : (cliquez dessus)

[main.c](#)

[Dico.h](#)

[Dico.c](#)

[Jeu.h](#)

[Jeu.c](#)

[Rail.h](#)

[Rail.c](#)

[File.h](#)

[File.c](#)

[Vecteur.h](#)

[Vecteur.c](#)

[Itemvecteur.h](#)

main.c

```

#pragma warning (disable: 4996)
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "Dico.h"

int main() {
    // XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX OUVERTURE DU DICTIONNAIRE
    XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
    FILE* f = fopen("ods4.txt", "r");
    if (f == NULL) {
        printf("Erreur d'ouverture du fichier\n");
        return;
    }
    size_t nbm = 0, nbc = 0;
    char mot[MAX_SAISI] = { 0 };
    int n;
    n = fscanf(f, "%29s", mot);

    Dico d;
    initDico(&d);

    Dico e;          //enregistre mot utilisées
    initDico(&e);

    while (n == 1) {
        ++nbm;
        nbc += strlen(mot);
        ajouterMot(&d, mot);
        n = fscanf(f, "%29s", mot);
    }
    if (ferror(f))
        printf("erreur - %s\n", strerror(errno));

    // XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX DEBUT DE LA PARTIE
    XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
    // Initialiser la file avec une capacité suffisante
    File p;
    initFile(&p, MAX_CHEVALETS);
    creerDeck(&p);
    //afficherPioche(p);
    melange(&p);
    //afficherPioche(p);

    Vecteur j1;
    initVecteur(&j1, MAX_C);
    Vecteur j2;
    initVecteur(&j2, MAX_C);

    for (int i = 0; i < MAX_C; i++) {
        distribution(&j1, &p);
        distribution(&j2, &p);
    }

    //afficherDeckJoueur(j1);
    //afficherDeckJoueur(j2);

    trier(j1.elements, j1.nbElements);
}

```



```

    trier(j2.elements, j2.nbElements);

    printf("1 : ");
    afficherDeckJoueur(j1);
    //afficherPioche(p);
    printf("2 : ");
    afficherDeckJoueur(j2);
    //afficherPioche(p);
    printf("\n");

    //JEU
    ItemV entree[MAX_SAISI];
    Vecteur m1;
    Vecteur m2;
    initVecteur(&m1, MOT_DE_QUATRE);
    initVecteur(&m2, MOT_DE_QUATRE);

    while (1) {
        printf("1> ");
        if (fgets(entree, sizeof(entree), stdin) == NULL) {
            continue; // En cas d'erreur ou d'entrée vide, redemander une
saisie
        }

        // Supprimer le retour à la ligne si présent
        ItemV len = strlen(entree);
        if (len > 0 && entree[len - 1] == '\n') {
            entree[len - 1] = '\0';
        }

        // Vérifier si l'utilisateur a appuyé sur Entrée sans rien saisir
        if (strlen(entree) == 0) {
            continue; // Redemander une saisie
        }

        if (strlen(entree) == MOT_DE_QUATRE) {
            int a = rechercheJoueur(&j1, entree);
            if (a == 1) {
                int b = rechercheDico(&d, entree);
                if (b == 1 && !rechercheDico(&e, entree)) {
                    supprimerLettre(&j1, entree);
                    ajoutMot(&m1, entree);
                    ajouterMot(&e, entree); //enregistre mot utilisée
                    break;
                }
            }
        }
    }

    while (1) {
        printf("2> ");
        if (fgets(entree, sizeof(entree), stdin) == NULL) {
            continue; // En cas d'erreur ou d'entrée vide, redemander une
saisie
        }

        // Supprimer le retour à la ligne si présent
        ItemV len = strlen(entree);
        if (len > 0 && entree[len - 1] == '\n') {
            entree[len - 1] = '\0';
        }
    }

```

```

    // Vérifier si l'utilisateur a appuyé sur Entrée sans rien saisir
    if (strlen(entree) == 0) {
        continue; // Redemander une saisie
    }

    if (strlen(entree) == MOT_DE_QUATRE) {
        int a = rechercheJoueur(&j2, entree);
        if (a == 1) {
            int b = rechercheDico(&d, entree);
            if (b == 1 && !rechercheDico(&e, entree)) {
                supprimerLettre(&j2, entree);
                ajoutMot(&m2, entree);
                ajouterMot(&e, entree);
                break;
            }
        }
    }
}
printf("\n");
//afficheDico(&e);

// XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX AFFICHAGE SITUATION COURANTE DU
DEBUT XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Numero num1; //Joueur
Numero num2; //Joueur
num1.chiffre = 1;
num2.chiffre = 2;

printf("%d : ", num1.chiffre);
//afficherDeckJoueur(j1);
afficherDeckJoueur(j1);
printf("%d : ", num2.chiffre);
afficherDeckJoueur(j2);

Vecteur Rail;
initVecteur(&Rail, MAX_R);
MisEnPlaceRail(&Rail, &m1, &m2);
printf("R : ");
afficheRail(&Rail);

Vecteur InverseRail;
initVecteur(&InverseRail, MAX_R);
//copieRail(&InverseRail, &Rail);
//swap(&InverseRail);
printf("V : ");
copieRail(&InverseRail, &Rail);
swap(&InverseRail);
afficheRail(&InverseRail);

// XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX CHANGE CELUI QUI COMMENCE EN FONCTION DU
MOT DE 4 LETTRES LE PLUS PETIT XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
ChangeNb(&num1, &num2, &m1, &m2); //Modification du courant avant le
vrai jeu
AlternerJoueur(&j1, &j2, &m1, &m2); //alterner joueur aussi

//afficherDeckJoueur(j1);
//afficherDeckJoueur(j2);
//afficherPioche(p);

```

```

//Cas échéant
Vecteur j1s;
Vecteur j2s;
Vecteur Rs;
Vecteur Vs;
ItemV MotPrecedent[MAX_SAISI] = "";
initVecteur(&j1s, MAX_C); //Ca peut être plus
initVecteur(&j2s, MAX_C);
initVecteur(&Rs, MAX_R);
initVecteur(&Vs, MAX_R);
AlternerJoueur(&j1s, &j2s, &m1, &m2); //alterner joueur pour
précédent aussi
copie(&j1s, &j1);
copie(&j2s, &j2);
copie(&Rs, &Rail);
copie(&Vs, &InverseRail);

// XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX JEU AVEC DIFFERENTS CAS
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
printf("\n");
while (taille(&j1) > 0 && taille(&j2) > 0) {
    // AU TOUR DU PREMIER JOUEUR QUI JOUE
    while (1) {
        ItemV saisi[MAX_SAISI];
        ItemV sansP[MAX_SAISI] = "";
        ItemV dansP[MAX_SAISI] = "";
        ItemV horsP[MAX_SAISI] = "";
        printf("%d> ", num1.chiffre);
        fgets(saisi, sizeof(saisi), stdin); // Utilisation de fgets
pour lire toute la ligne
        // Supprimer le retour à la ligne si présent à la fin de la
chaîne
        ItemV len = strlen(saisi);
        if (len > 0 && saisi[len - 1] == '\n') {
            saisi[len - 1] = '\0'; // Retirer le '\n' qui reste après
fgets
        }
        if (strcmp(saisi, "exit") == 0) {
            break; // Sort de la boucle si "exit" est saisi
        }
        if (strlen(saisi) == 0) {
            continue; // Redemander une saisie
        }
        // JOUEUR FORME MOT À UNE EXTRÉMITÉ DU RAIL RECTO R
        if (saisi[0] == 'R' && saisi[1] == ' ') {
            int a = jouerCoup(&d, &j1, saisi, &Rail); //Vérifie s'il
le joueur peut jouer
            if (a == 1) {
                lireMotSansParentheses(saisi, sansP, DEBUT_MOT);
                HorsParentheses(saisi, horsP, DEBUT_MOT);
                Init(&j1s, &j2s, &Rs, &Vs);
                copie(&j1s, &j1); //Stocke les
valeurs
                copie(&j2s, &j2);
                copie(&Rs, &Rail);
                copie(&Vs, &InverseRail);
                int b = editRail(&Rail, saisi, &j2); //On modifie
les valeurs
                if (b == 1 && !rechercheDico(&e, sansP)) {
                    ajouterMot(&e, sansP);
                    supprimerLettre(&j1, horsP);

```

```

strcpy(MotPrecedent, sansP);
updRail(&InverseRail, &Rail);
if (strlen(sansP) == MAX_MOT) {
    printf("\n");
    situationActu(&j1, &j2, &Rail, &InverseRail,
&num1, &num2);
    // CAS SI MOT DE 8 LETTRES : JOUEUR ENLÈVE UN
    DE SES CHEVALETS

    char L[MAX_LETTRE];
    int x = 0;
    do {
        // Demander une saisie utilisateur
        printf("-> ", num1.chiffre);
        fgets(L, sizeof(L), stdin);
        char lon = strlen(L);
        if (lon > 0 && L[lon - 1] == '\n') {
            L[lon - 1] = '\0'; // Retirer le '\n'
qui reste après fgets
        }
        // Vérifier si la saisie est correcte
        if (strlen(L) == 1) {
            x = rechercheJoueur(&j1, L);
        }
        // Si l'entrée est invalide, on redemande
la saisie
    } while (strlen(L) == 0 || x != 1);
    // Si le joueur existe, on ajoute la lettre à
la pioche et on la retire du joueur
    if (x == 1) {
        ajoutPioche(&p, L); // Fonction remet dans
pioche
        supprimerLettre(&j1, L);
    }
    break;
}
break;
}
}
// CAS JOUEUR FORME MOT À UNE EXTRÉMITÉ DU RAIL RECTO V
else if (saisi[0] == 'V' && sais[1] == ' ') {
    int b = jouerCoup(&d, &j1, sais, &InverseRail);
    if (b == 1) {
        lireMotSansParentheses(sais, sansP, DEBUT_MOT);
        Init(&j1s, &j2s, &Rs, &Vs);
        copie(&j1s, &j1); //Stocke les
valeurs
        copie(&j2s, &j2);
        copie(&Rs, &Rail);
        copie(&Vs, &InverseRail);
        HorsParentheses(sais, horsP, DEBUT_MOT);
        int b = editRail(&InverseRail, sais, &j2);
        if (b == 1 && !rechercheDico(&e, sansP)) {
            ajouterMot(&e, sansP);
            supprimerLettre(&j1, horsP);
            strcpy(MotPrecedent, sansP);
            updRail(&Rail, &InverseRail);
            if (strlen(sansP) == MAX_MOT) {
                printf("\n");
                situationActu(&j1, &j2, &Rail, &InverseRail,
&num1, &num2);

```

```

// SI MOT DE 8 LETTRES : JOUEUR ENLÈVE UN DE
SES CHEVALETS

char L[MAX_LETTRE];
int x = 0;
do {
    // Demander une saisie utilisateur
    printf("-%d> ", num1.chiffre);
    fgets(L, sizeof(L), stdin);
    char lon = strlen(L);
    if (lon > 0 && L[lon - 1] == '\n') {
        L[lon - 1] = '\0'; // Retirer le '\n'
    }
    // Vérifier si la saisie est correcte
    if (strlen(L) == 1) {
        x = rechercheJoueur(&j1, L);
    }
    // Si l'entrée est invalide, on redemande
    } while (strlen(L) == 0 || x != 1);
    // Si le joueur existe, on ajoute la lettre à
    la pioche et on la retire du joueur
    if (x == 1) {
        ajoutPioche(&p, L); // Fonction remet dans
        pioche
        supprimerLettre(&j1, L);
    }
    break;
}
break;
}
}
// CAS SIGNALER UN MOT QUI AURAIT PU ÊTRE JOUÉ PRÉCÉDEMMENT PAR
L'ADVERSAIRE
if (strlen(MotPrecedent) < MAX_MOT) {
    if ((saisi[0] == 'r' || saisi[0] == 'v') && saisi[1] == '
') {
        // SUR LE RAIL RECTO R
        if (saisi[0] == 'r') {
            lireMotSansParentheses(saisi, sansP, DEBUT_MOT);
            if (strlen(sansP) == MAX_MOT) {
                int a = jouerCoup(&d, &j2s, sais, &Rs);
                if (a == 1) {
                    int b = VerifRail(&Rs, sais);
                    if (b == 1 && !rechercheDico(&e, sansP)) {
                        printf("\n");
                        situationActu(&j1, &j2, &Rail,
                        &InverseRail, &num1, &num2);
                    }
                }
            }
        }
        // SI MOT DE 8 LETTRES : JOUEUR AYANT
        SIGNALÉ ENLÈVE UN DE SES CHEVALETS
        un caractère + '\0'
        char L[MAX_LETTRE]; // Taille 2 pour
        int x = 0;
        do {
            // Demander une saisie utilisateur
            printf("-%d> ", num1.chiffre);
            fgets(L, sizeof(L), stdin);
            char lon = strlen(L);
            if (lon > 0 && L[lon - 1] == '\n')

```

```

le '\n' qui reste après fgets
correcte
redemande la saisie
lettre à la pioche et on la retire du joueur
remet dans pioche

L[lon - 1] = '\0'; // Retirer
}
// Vérifier si la saisie est
if (strlen(L) == 1) {
    x = rechercheJoueur(&j1, L);
}
// Si l'entrée est invalide, on
} while (strlen(L) == 0 || x != 1);
// Si le joueur existe, on ajoute la
if (x == 1) {
    ajoutPioche(&p, L); // Fonction
    supprimerLettre(&j1, L);
}
ajouterMot(&e, sansP);
printf("\n");
updRail(&InverseRail, &Rail);
situationActu(&j1, &j2, &Rail,
&InverseRail, &num1, &num2);
}
}
}
// SUR LE RAIL VERSO V
else { //Cas de 'v'
    lireMotSansParentheses(saisi, sansP, DEBUT_MOT);
    if (strlen(sansP) == MAX_MOT) {
        int a = jouerCoup(&d, &j2s, sais, &Vs);
        if (a == 1) {
            int b = VerifRail(&Vs, sais);
            if (b == 1 && !rechercheDico(&e, sansP)) {
                printf("\n");
                situationActu(&j1, &j2, &Rail,
&InverseRail, &num1, &num2);
                // SI MOT DE 8 LETTRES : JOUEUR AYANT
                SIGNALÉ ENLÈVE UN DE SES CHEVALETS
                char L[MAX_LETTRE];
                int x = 0;
                do {
                    // Demander une saisie utilisateur
                    printf("-%d> ", num1.chiffre);
                    fgets(L, sizeof(L), stdin);
                    char lon = strlen(L);
                    if (lon > 0 && L[lon - 1] == '\n')
{
                    L[lon - 1] = '\0'; // Retirer
                    }
                    // Vérifier si la saisie est
                    if (strlen(L) == 1) {
                        x = rechercheJoueur(&j1, L);
                    }
                    // Si l'entrée est invalide, on
                } while (strlen(L) == 0 || x != 1);

```

```

// Si le joueur existe, on ajoute la
lettre à la pioche et on la retire du joueur
    if (x == 1) {
        ajoutPioche(&p, L); // Fonction
remet dans pioche
        supprimerLettre(&j1, L);
    }
    ajouterMot(&e, sansP);
    printf("\n");
    updRail(&Rail, &InverseRail);
    situationActu(&j1, &j2, &Rail,
&InverseRail, &num1, &num2);
}
}
}
}
}
// CAS OÙ JOUEUR VEUT ÉCHANGER UN DE SES CHEVALETS AVEC UN DE
LA PIOCHE
ItemV echgL[MAX_SAISI] = "";
if (saisi[0] == '-' && sais[1] == ' ') {
    if (strlen(saisi) == NB_CARACTERES_STOCKE) {
        traiterLettre(echgL, sais, DEBUT_MOT);
        int a = rechercheJoueur(&j1, echgL);
        if (a == 1) {
            echgChevalet(&p, &j1, echgL);
            //afficherPioche(p);
            strcpy(MotPrecedent, " ");
            break;
        }
    }
}
}
printf("\n");
if (j1.nbElements == 0 || j2.nbElements == 0) {
    break;
}
trier(j1.elements, j1.nbElements);
trier(j2.elements, j2.nbElements);
situationActu(&j1, &j2, &Rail, &InverseRail, &num1, &num2);

//printf("Précédent:\n");
//afficherDeckJoueur(j1s);
//afficherDeckJoueur(j2s);
//afficheRail(&Rs);
//afficheRail(&Vs);

// AU TOUR DU SECOND JOUEUR QUI JOUE
while (1) {
    ItemV sais[0] = "";
    ItemV sansP[MAX_SAISI] = "";
    ItemV dansP[MAX_SAISI] = "";
    ItemV horsP[MAX_SAISI] = "";
    printf("%d> ", num2.chiffre);
    fgets(sais, sizeof(sais), stdin);

    ItemV len = strlen(sais);
    if (len > 0 && sais[len - 1] == '\n') {

```

```

    saisi[len - 1] = '\\0'; // Retirer le '\\n' qui reste après
fgets
}
if (strcmp(saisi, "exit") == 0) {
    break; // Sort de la boucle si "exit" est saisi
}
if (strlen(saisi) == 0) {
    continue; // Redemander une saisie
}
// JOUEUR FORME MOT À UNE EXTRÉMITÉ DU RAIL RECTO R
if (saisi[0] == 'R' && saisi[1] == ' ') {
    int a = jouerCoup(&d, &j2, saisi, &Rail);
    if (a == 1) {
        lireMotSansParentheses(saisi, sansP, DEBUT_MOT);
        HorsParentheses(saisi, horsP, DEBUT_MOT);
        Init(&j1s, &j2s, &Rs, &Vs);
        copie(&j1s, &j1);
        copie(&j2s, &j2);
        copie(&Rs, &Rail);
        copie(&Vs, &InverseRail);
        int b = editRail(&Rail, saisi, &j1);
        if (b == 1 && !rechercheDico(&e, sansP)) {
            ajouterMot(&e, sansP);
            supprimerLettre(&j2, horsP);
            strcpy(MotPrecedent, sansP);
            updRail(&InverseRail, &Rail);
            if (strlen(sansP) == MAX_MOT) {
                printf("\\n");
                situationActu(&j1, &j2, &Rail, &InverseRail,
&num1, &num2);
                // CAS SI MOT DE 8 LETTRES : JOUEUR ENLÈVE UN
DE SES CHEVALETS
                char L[MAX_LETTRE];
                int x = 0;
                do {
                    // Demander une saisie utilisateur
                    printf("-%d> ", num2.chiffre);
                    fgets(L, sizeof(L), stdin);
                    char lon = strlen(L);
                    if (lon > 0 && L[lon - 1] == '\\n') {
                        L[lon - 1] = '\\0'; // Retirer le '\\n'
qui reste après fgets
                    }
                    // Vérifier si la saisie est correcte
                    if (strlen(L) == 1) {
                        x = rechercheJoueur(&j2, L);
                    }
                    // Si l'entrée est invalide, on redemande
la saisie
                } while (strlen(L) == 0 || x != 1);
                // Si le joueur existe, on ajoute la lettre à
la pioche et on la retire du joueur
                if (x == 1) {
                    ajoutPioche(&p, L); // Fonction remet dans
pioche
                    supprimerLettre(&j2, L);
                }
                break;
            }
            break;
        }
    }
}

```



```

    }
}
// JOUEUR FORME MOT À UNE EXTRÉMITÉ DU RAIL VERSO V
else if (saisi[0] == 'V' && sais[1] == ' ')
{
    //Inversé les Rails
    int b = jouerCoup(&d, &j2, sais, &InverseRail);
    if (b == 1) {
        lireMotSansParentheses(sais, sansP, DEBUT_MOT);
        HorsParentheses(sais, horsP, DEBUT_MOT);
        Init(&j1s, &j2s, &Rs, &Vs);
        copie(&j1s, &j1);
        copie(&j2s, &j2);
        copie(&Rs, &Rail);
        copie(&Vs, &InverseRail);
        int b = editRail(&InverseRail, sais, &j1);    ///booléen
INT!!!!

        if (b == 1 && !rechercheDico(&e, sansP)) {
            ajouterMot(&e, sansP);
            supprimerLettre(&j2, horsP);
            strcpy(MotPrecedent, sansP);
            updRail(&Rail, &InverseRail);
            if (strlen(sansP) == MAX_MOT) {
                printf("\n");
                situationActu(&j1, &j2, &Rail, &InverseRail,
&num1, &num2);

                // CAS SI MOT DE 8 LETTRES : JOUEUR ENLÈVE UN
DE SES CHEVALETS

                char L[MAX_LETTRE];
                int x = 0;
                do {
                    // Demander une saisie utilisateur
                    printf("-%d> ", num2.chiffre);
                    fgets(L, sizeof(L), stdin);
                    char lon = strlen(L);
                    if (lon > 0 && L[lon - 1] == '\n') {
                        L[lon - 1] = '\0';    // Retirer le '\n'
qui reste après fgets

                    }
                    // Vérifier si la saisie est correcte
                    if (strlen(L) == 1) {
                        x = rechercheJoueur(&j2, L);
                    }
                    // Si l'entrée est invalide, on redemande
la saisie

                } while (strlen(L) == 0 || x != 1);
                // Si le joueur existe, on ajoute la lettre à
la pioche et on la retire du joueur
                if (x == 1) {
                    ajoutPioche(&p, L);    // Fonction remet dans
pioche

                    supprimerLettre(&j2, L);
                }
                break;
            }
            break;
        }
    }
}
// CAS SIGNALER UN MOT QUI AURAIT PU ÊTRE JOUÉ PRÉCÉDEMMENT PAR
L'ADVERSAIRE
if (strlen(MotPrecedent) < MAX_MOT) {

```

```

        if ((saisi[0] == 'r' || sais[0] == 'v') && sais[1] == '
    ') {
        // SUR LE RAIL RECTO R
        if (saisi[0] == 'r') {
            lireMotSansParentheses(saisi, sansP, DEBUT_MOT);
            if (strlen(sansP) == MAX_MOT) {
                int a = jouerCoup(&d, &jls, sais, &Rs);
                if (a == 1) {
                    int b = VerifRail(&Rs, sais);
                    if (b == 1 && !rechercheDico(&e, sansP)) {
                        printf("\n");
                        situationActu(&j1, &j2, &Rail,
&InverseRail, &num1, &num2);
                        // SI MOT DE 8 LETTRES : JOUEUR AYANT
SIGNALÉ ENLÈVE UN DE SES CHEVALETS
                        char L[MAX_LETTRE];
                        int x = 0;
                        do {
                            // Demander une saisie utilisateur
                            printf("-%d> ", num2.chiffre);
                            fgets(L, sizeof(L), stdin);
                            char lon = strlen(L);
                            if (lon > 0 && L[lon - 1] == '\n')
{
                                L[lon - 1] = '\0'; // Retirer
le '\n' qui reste après fgets
                                // Vérifier si la saisie est
correcte
                                if (strlen(L) == 1) {
                                    x = rechercheJoueur(&j2, L);
                                }
                                // Si l'entrée est invalide, on
redemande la saisie
                                } while (strlen(L) == 0 || x != 1);
                                // Si le joueur existe, on ajoute la
lettre à la pioche et on la retire du joueur
                                if (x == 1) {
                                    ajoutPioche(&p, L); // Fonction
remet dans pioche
                                    supprimerLettre(&j2, L);
                                }
                                ajouterMot(&e, sansP);
                                printf("\n");
                                updRail(&InverseRail, &Rail);
                                situationActu(&j1, &j2, &Rail,
&InverseRail, &num1, &num2);
                            }
                        }
                    }
                }
            }
        // SUR LE RAIL RECTO R
        else { //Cas de 'v'
            lireMotSansParentheses(saisi, sansP, DEBUT_MOT);
            if (strlen(sansP) == MAX_MOT) {
                int a = jouerCoup(&d, &jls, sais, &Vs);
                if (a == 1) {
                    int b = VerifRail(&Vs, sais);
                    if (b == 1 && !rechercheDico(&e, sansP)) {
                        printf("\n");

```

```

    &InverseRail, &num1, &num2);
    SIGNALÉ ENLÈVE UN DE SES CHEVALETS
    un caractère + '\0'

    situationActu(&j1, &j2, &Rail,
    // SI MOT DE 8 LETTRES : JOUEUR AYANT
    char L[MAX_LETTRE]; // Taille 2 pour
    int x = 0;
    do {
        // Demander une saisie utilisateur
        printf("-%d> ", num2.chiffre);
        fgets(L, sizeof(L), stdin);
        char lon = strlen(L);
        if (lon > 0 && L[lon - 1] == '\n')
            L[lon - 1] = '\0'; // Retirer
        // Vérifier si la saisie est
        if (strlen(L) == 1) {
            x = rechercheJoueur(&j2, L);
        }
        // Si l'entrée est invalide, on
        } while (strlen(L) == 0 || x != 1);
        // Si le joueur existe, on ajoute la
        lettre à la pioche et on la retire du joueur
        if (x == 1) {
            ajoutPioche(&p, L); // Fonction
            supprimerLettre(&j2, L);
        }
        ajouterMot(&e, sansP);
        printf("\n");
        updRail(&Rail, &InverseRail);
        situationActu(&j1, &j2, &Rail,
    &InverseRail, &num1, &num2);
    }
    }
    }
    }
    }
    // CAS OÙ JOUEUR VEUT ÉCHANGER UN DE SES CHEVALETS AVEC UN DE
    LA PIOCHE
    ItemV echgL[MAX_SAISI] = "";
    if (saisi[0] == '-' && sais[1] == ' ') {
        if (strlen(saisi) == NB_CARACTERES_STOCKE) {
            traiterLettre(echgL, sais, DEBUT_MOT);
            int a = rechercheJoueur(&j2, echgL);
            if (a == 1) {
                echgChevalet(&p, &j2, echgL);
                //afficherPioche(p);
                strcpy(MotPrecedent, " ");
                break;
            }
        }
    }
    }
    printf("\n");

```

```

        if (j1.nbElements == 0 || j2.nbElements == 0) {
            break;
        }

        trier(j1.elements, j1.nbElements);
        trier(j2.elements, j2.nbElements);
        situationActu(&j1, &j2, &Rail, &InverseRail, &num1, &num2);

    }
    //Fin de la boucle while principale

    //Affiche résultat
    //printf("-----");

    // XXXXXXXXXXXXXXXXXXXXXXXXXXXX LIBÉRATION DE LA MÉMOIRE ALLOUÉE
    XXXXXXXXXXXXXXXXXXXXXXXXXXXX
    detruireVecteur(&j1);
    detruireVecteur(&j2);
    detruireVecteur(&j1s);
    detruireVecteur(&j2s);
    detruireVecteur(&Rail);
    detruireVecteur(&InverseRail);
    detruireVecteur(&Rs);
    detruireVecteur(&Vs);
    detruireVecteur(&m2);
    detruireVecteur(&m1);
    detruireFile(&p);
    libererDico(&d);
    libererDico(&e);
    // XXXXXXXXXXXXXXXXXXXXXXXXXXXX FERMETURE DU DICTIONNAIRE
    XXXXXXXXXXXXXXXXXXXXXXXXXXXX
    fclose(f);
    return 0;
}

```

Dico.h

```

#pragma once

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "Jeu.h"

/*Structures de données et fonctions permettant l'implémentation et la
vérification des mots*/

/**
 * @brief Composant de dictionnaire contenant un tableau dynamique de mots.
 * Elle inclut également un champ pour la gestion de la quantité totale de
 mots dans le dictionnaire.
 */
typedef struct {
    char** mots; // Pointeur vers un tableau de chaînes de caractères
 (tableau de mots)
    size_t qte; // Quantité de mot dans le dictionnaire
}Dico;

```

```

/**
 * @brief Initialise un dictionnaire vide.
 * En définissant la quantité de mots à 0
 * et le tableau de mots à NULL.
 * Après son utilisation, la mémoire occupée par l'enregistrement doit être
libérée
 * en invoquant la fonction @ref libererDico.
 * @param[in,out] d Pointeur vers la structure Dico à initialiser.
 * La fonction modifie le contenu de cette structure en initialisant ses
champs.
 */
void initDico(Dico* d);

/**
 * @brief Ajoute un mot au dictionnaire en allouant dynamiquement de la
mémoire
 * pour le mot et en augmentant la quantité de mots dans la structure.
 * Elle vérifie également que le mot n'est pas vide et que les allocations
mémoire réussissent.
 * @param[in, out] d Pointeur vers le dictionnaire.
 * La fonction modifie le dictionnaire en ajoutant un mot à la structure.
 * @param[in] mot Le mot à ajouter dans le dictionnaire.
 */
void ajouterMot(Dico* d, const char* mot);

/**
 * @brief Affiche le contenu du dictionnaire.
 * Cette fonction parcourt tous les mots du dictionnaire et les affiche
 * @param[in] d Pointeur vers le dictionnaire à afficher.
 * La fonction n'affecte pas la structure mais utilise ses données pour
afficher les mots.
 */
void afficheDico(Dico* d);

/**
 * @brief Recherche un mot dans le dictionnaire.
 * @param[in] d Pointeur vers le dictionnaire dans lequel rechercher le
mot.
 * La fonction n'affecte pas la structure, mais l'utilise pour rechercher
le mot.
 * @param[in] mot Le mot à rechercher dans le dictionnaire.
 * La fonction compare ce mot aux mots du dictionnaire pour déterminer s'il
est présent.
 * @return[out] 1 si le mot est trouvé, 0 si le mot n'est pas trouvé.
 */
int rechercheDico(Dico* d, const char* mot);

/**
 * @brief Libère la mémoire allouée pour le dictionnaire.
 * Cette fonction désalloue toute la mémoire utilisée pour stocker les mots
du dictionnaire,
 * ainsi que le tableau de pointeurs lui-même. Elle réinitialise également
la structure du
 * dictionnaire en mettant la quantité à 0 et le tableau à NULL.
 * @param[in, out] d Pointeur vers le dictionnaire à libérer.
 */
void libererDico(Dico* d);

/**
 * @brief Validation du coup qui sera jouer par un joueur.

```

```

* Vérification de la saisie.
* @param[in,out] d L'adresse du dictionnaire.
* @param[in,out] j L'adresse d'un vecteur. ( Joueur )
* @param[in] entree Une chaîne de caractère lors du saisi
* @param[in,out] r L'adresse d'un vecteur. ( Rail )
* @return 1 si le coup peut être joué par le joueur et 0 en cas d'échec.
*/
int jouerCoup(Dico* d, Vecteur* j, const ItemV* entree, Vecteur* r);

```

Dico.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "Dico.h"

/**
 * @file Dico.h
 * @brief Composant de dictionnaire contenant un tableau dynamique de mots.
 * Elle inclut également un champ pour la gestion de la quantité totale de
 * mots dans le dictionnaire.
 */

void initDico(Dico* d) {
    d->qte = 0;
    d->mots = NULL;
}

void ajouterMot(Dico* d, const char* mot) {
    if (mot == NULL || strlen(mot) == 0) {
        printf("Mot invalide.\n");
        return;
    }

    d->mots = (char**)realloc(d->mots, (d->qte + 1) * sizeof(char*));
    if (d->mots == NULL) {
        printf("Erreur d'allocations mémoire dico\n");
        return;
    }
    d->mots[d->qte] = (char*)malloc((strlen(mot) + 1) * sizeof(char));
    if (d->mots[d->qte] == NULL) {
        printf("Erreur d'allocations mémoire pour le mot\n");
        return;
    }

    if (strcpy_s(d->mots[d->qte], strlen(mot) + 1, mot) != 0) {
        printf("Erreur de copie du mot\n");
        return;
    }
    d->qte++;
}

void afficheDico(Dico* d) {
    for (size_t i = 0; i < d->qte; ++i) {
        printf("%zu: %s\n", i + 1, d->mots[i]);
    }
}

int rechercheDico(Dico* d, const char* mot) {
    if (mot == NULL || strlen(mot) == 0) {

```

```

        printf("erreur (rechercheDico) \n");
        return 0;
    }
    for (size_t i = 0; i < d->qte; ++i) {           //Recherche du mot dans le
dictionnaire
        if (strcmp(d->mots[i], mot) == 0) {         //On regarde dans dico si
y'a le mot
            return 1;                               //retourne 1 si le mot est
trouvé
        }
    }
    return 0;    //retourne 0 si le mot n'est pas trouvé
}

// Fonction pour désallouer la mémoire du dictionnaire
void libererDico(Dico* d) {
    // Libérer la mémoire pour chaque mot dans le dictionnaire
    for (size_t i = 0; i < d->qte; ++i) {
        free(d->mots[i]);
    }

    // Libère le tableau de pointeurs vers les mots
    free(d->mots);

    // Réinitialiser les champs du dictionnaire
    d->qte = 0;
    d->mots = NULL;
}

int jouerCoup(Dico* d, Vecteur* j, const ItemV* entree, Vecteur* r) {
    ItemV mot[MAX_SAISI] = "";
    ItemV rail[MAX_SAISI] = "";
    ItemV chevalet[MAX_SAISI] = "";

    dansParentheses(entree, rail); //Ce qui est dans la parentheses puis
le comparer au Rail
    lireMotSansParentheses(entree, mot, DEBUT_MOT);
    HorsParentheses(entree, chevalet, DEBUT_MOT);

    //printf("r: %s\n", rail);           //ce qui est dans la parenthese
    //printf("mot: %s\n", mot);         //Le mot former
    //printf("c: %s\n", chevalet);     //affiche les chevalets jouer par le
joueur

    if (strlen(mot) <= MAX_MOT && strlen(rail) >= DEBUT_MOT &&
strlen(mot) > strlen(rail)) {
        int a = rechercheJoueur(j, chevalet);
        if (a == 1) {
            int b = DansRail(r, rail);
            if (b == 1) {
                //printf("dansR\n");    //Pour vérifier s'il est dans Rail
                int c = rechercheDico(d, mot);
                if (c == 1) {
                    return 1;
                }
            }
        }
    }
    return 0;
}

```

Jeu.h

```

#pragma once

#include "Rail.h"

/*Fonctions et structures de données gérant la mise en route d'une partie
de
RECTO-VERSO*/

typedef struct {
    unsigned int chiffre; // Chiffre entier non signé.
}Numero;

/**
 * @brief Trie les chevauxets, c'est-à-dire les lettres d'un joueur.
 * @param[in,out] chevalet Le chevalet à trier.
 * @param[in] taille Le nombre de lettres que possède le joueur.
 */
void trier(char* chevalet, int taille);

/**
 * @brief Affiche le contenu des chevauxets du joueur.
 * @param[in] v Le vecteur contenant les lettres, c'est-à-dire les
chevalets du joueur.
 */
void afficherDeckJoueur(Vecteur v);

/**
 * @brief Recherche d'une chaîne de caractère dans le vecteur.
 * Vérifie qu'un joueur a toutes les lettres composants le mot
 * @param[in,out] j L'adresse du vecteur.
 * @param[in] c Une chaîne d'élément qui est le mot formé par le joueur
 * @return 1 si le mot est trouvée et 0 cas d'échec.
 */
int rechercheJoueur(Vecteur* j, const char* c);

/**
 * @brief Supprimer un élément dans un vecteur.
 * @param[in,out] j L'adresse d'un vecteur
 * @param[in] c L'élément à retiré.
 */
void supprimerLettre(Vecteur* j, const ItemV* c);

/**
 * @brief Ajout d'un élément dans un vecteur. ( une chaîne de caractère )
 * @param[in,out] v L'adresse d'un vecteur
 * @param[in] m L'élément à ajouter.
 */
void ajoutMot(Vecteur* v, const ItemV* m);

/**
 * @brief Copie les éléments d'un vecteur dans un autre vecteur.
 * @param[out] v1 L'adresse du vecteur cible.
 * @param[in] v2 L'adresse du vecteur source.
 */
void copie(Vecteur* v1, Vecteur* v2);

/**

```



```

* @brief Met à jour le numéro courant du joueur.
* Modifie les numéros en fonctions du mot créer au début du jeu.
* @param[out] n1 Numéro à modifier
* @param[out] n2 Numéro à modifier
* @param[in] m1 L'élément du vecteur à comparer.
* @param[in] m2 L'élément du vecteur à comparer.
*/
void ChangeNb(Numero* n1, Numero* n2, const Vecteur* m1, const Vecteur*
m2);

/**
* @brief Alterne les éléments de deux vecteurs.
* Alterne les joueurs en fonction du mot créer au début.
* @param[in,out] j1 L'adresse d'un vecteur.
* @param[in,out] j2 L'adresse d'un vecteur.
* @param[in] m1 L'adresse d'un vecteur contenant l'élément à comparer.
* @param[in] m2 L'adresse d'un vecteur contenant l'élément à comparer.
*/
void AlternerJoueur(Vecteur* j1, Vecteur* j2, const Vecteur* m1, const
Vecteur* m2);

/**
* @brief Affiche les éléments des vecteurs.
* Affiche la situation actuelle du Jeu. ( Joueurs et rails )
* @param[in,out] j1 L'adresse d'un vecteur.
* @param[in,out] j2 L'adresse d'un vecteur.
* @param[in,out] r L'adresse d'un vecteur.
* @param[in,out] v L'adresse d'un vecteur.
* @param[in,out] n1 L'adresse d'un numéro.
* @param[in,out] n2 L'adresse d'un numéro.
*/
void situationActu(Vecteur* j1, Vecteur* j2, Vecteur* r, Vecteur* v,
Numero* n1, Numero* n2);

/**
* @brief Traite l'élément fait lors de la saisi.
* Supprime les caractères inutiles.
* @param[in,out] lettres Lettres sont traités.
* @param[in] saisi L'élément de la saisi.
* @param[in] debut La position initiale pour le traitement.
*/
void traiterLettre(ItemV* lettres, ItemV* saisi, int debut);

/**
* @brief Echange d'élément d'une file et d'un vecteur.
* Echange d'une lettre dans le chevalet du joueur avec la pioche.
* @param[in,out] f L'adresse d'une file. ( Pioche )
* @param[in,out] j L'adresse d'un vecteur. ( Joueur )
* @param[in] lettre L'élément à échanger avec une file.
*/
void echgChevalet(File* f, Vecteur* j, ItemV* lettre);

/**
* @brief Initialisation des vecteurs.
* On réinitialise les vecteurs et on va stocker les éléments précédents.
* @param[in,out] j1 L'adresse d'un vecteur.
* @param[in,out] j2 L'adresse d'un vecteur.
* @param[in,out] r L'adresse d'un vecteur.
* @param[in,out] v L'adresse d'un vecteur.
*/
void Init(Vecteur* j1, Vecteur* j2, Vecteur* r, Vecteur* v);

```

Jeu.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <assert.h>
#include "Jeu.h"

/*Fonctions et structures de données gérant la mise en route
d'une partie de RECTO-VERSO*/

void trier(char* chevalet, int taille) {
    for (int i = 0; i < taille - 1; ++i) {
        for (int j = i + 1; j < taille; ++j) {
            if (chevalet[i] > chevalet[j]) {
                char tmp = chevalet[i];
                chevalet[i] = chevalet[j];
                chevalet[j] = tmp;
            }
        }
    }
}

void afficherDeckJoueur(Vecteur v) {
    for (int i = 0; i < v.nbElements; i++) {
        printf("%c", v.elements[i]); // Afficher chaque lettre dans la
file
    }
    printf("\n");
}

int rechercheJoueur(Vecteur* j, const char* mot) {
    if (mot == NULL || strlen(mot) == 0) {
        printf("Mot invalide.\n");
        return 0;
        printf("N.\n");
    }

    // Crée un tableau temporaire pour marquer les lettres utilisées
    //int lettresUtilisees[MAX_C] = { 0 }; // MAX_C est la taille max des
chevalets du joueur
    size_t motLength = strlen(mot);

    int* lettresUtilisees = (int*)calloc(j->nbElements, sizeof(int)); //
Dynamique selon j->nbElements
    if (!lettresUtilisees) {
        printf("Erreur mémoire.\n");
        return 0;
    }

    // Parcourt chaque lettre du mot
    for (size_t k = 0; k < motLength; ++k) {
        int trouve = 0;

        // Parcourt les lettres du chevalet pour chercher une
correspondance non utilisée

```

```

    for (int x = 0; x < j->nbElements; ++x) {
        if (!lettresUtilisees[x] && j->elements[x] == mot[k]) {
            lettresUtilisees[x] = 1; // Marque la lettre comme utilisée
            trouve = 1;              // Lettre trouvée
            break;                   // Sort de la boucle interne
        }
    }

    if (trouve == 0) {
        free(lettresUtilisees);
        return 0; // Lettre introuvable ou déjà utilisée
    }
}

free(lettresUtilisees); // Libère la mémoire après usage
return 1;
}

void supprimerLettre(Vecteur* j, const ItemV* c) {
    if (c == NULL || strlen(c) == 0) {
        printf("Lettre invalide.\n");
        return;
    }

    for (int k = 0; k < strlen(c); k++) {
        for (int i = 0; i < j->nbElements; i++) {
            if (j->elements[i] == c[k]) {
                supprimer(j, i);
                break;
            }
        }
    }
}

void ajoutMot(Vecteur* v, const ItemV* m) {
    for (int i = 0; i < strlen(m); i++) {
        if (!ajouter(v, m[i])) {
            printf("Erreur d'ajout\n");
            break;
        }
    }
}

void copie(Vecteur* v1, Vecteur* v2) {
    for (int i = 0; i < v2->nbElements; i++) {
        if (!ajouter(v1, v2->elements[i])) {
            printf("Erreur d'ajout\n");
            break;
        }
    }
}

//Changement de courant pour Joueur
void ChangeNb(Numero* n1, Numero* n2, const Vecteur* m1, const Vecteur* m2)
{
    int result = strcmp(m1->elements, m2->elements);
    if (result > 0) { //m1 avant m2, échanger
        int temp = n1->chiffre;
        n1->chiffre = n2->chiffre;
        n2->chiffre = temp;
    }
}

```

```

    }
}

void AlternerJoueur(Vecteur* j1, Vecteur* j2, const Vecteur* m1, const
Vecteur* m2) {
    int result = strcmp(m1->elements, m2->elements);
    if (result > 0) { //m1 avant m2, échanger
        Vecteur temp = *j1; //échanger les champs vecteurs j1 et j2
        *j1 = *j2;
        *j2 = temp;
    }
}

void situationActu(Vecteur* j1, Vecteur* j2, Vecteur* r, Vecteur* v,
Numero* n1, Numero* n2) {
    //fonction compare num.chiffre lequel est plus petit et inverser
    afficheDeckJoueur

    int a = 0;
    int b = 0;
    unsigned int num1 = n1->chiffre;
    unsigned int num2 = n2->chiffre;

    if (num1 == 1) {
        printf("%d : ", n1->chiffre);
        a = 1;
    }
    else {
        printf("%d : ", n2->chiffre);
        a = 0;
    }

    if (a == 1) {
        //int result = taille(j1);
        //retailer(j1, result);
        afficheDeckJoueur(*j1);
    }
    else {
        //int result = taille(j2);
        //retailer(j2, result);
        afficheDeckJoueur(*j2);
    }

    if (num2 == 2) {
        printf("%d : ", n2->chiffre);
        b = 1;
    }
    else {
        printf("%d : ", n1->chiffre);
        b = 0;
    }

    if (b == 1) {
        //int result = taille(j2);
        //retailer(j2, result);
        afficheDeckJoueur(*j2);
    }
    else if (b == 0) {
        //int result = taille(j1);
        //retailer(j1, result);
        afficheDeckJoueur(*j1);
    }
}

```

```

    }
    printf("R : ");
    afficheRail(r);
    printf("V : ");
    afficheRail(v);
    printf("\n");
}

void traiterLettre(ItemV* lettres, ItemV* saisi, int debut) {
    int idx = 0;

    for (int i = debut; saisi[i]!='\0'; i++) { //Parcours jusqu'à fin de
chaîne
        lettres[idx++] = saisi[i];
    }
    lettres[idx] = '\0';
}

void echgChevalet(File* f, Vecteur* j, ItemV* lettre) {
    ajoutMot(f, lettre);
    supprimerLettre(j, lettre);
    distribution(j, f);
}

void Init(Vecteur* j1, Vecteur* j2, Vecteur* r, Vecteur* v) {
    detruireVecteur(j1);
    detruireVecteur(j2);
    detruireVecteur(r);
    detruireVecteur(v);
    initVecteur(j1, MAX_C); //Ca peut être plus
    initVecteur(j2, MAX_C);
    initVecteur(r, MAX_R);
    initVecteur(v, MAX_R);
}

```

Rail.h

```

#pragma once

#include "File.h"

/*Fonctions permettant de gérer et vérifier les chevaux sur le rail*/

enum {
    DEBUT_MOT = 2, // Extraire l'élément à partir du 3 caractère

    MAX_SAISI = 30, // Maximum pour la saisi
    MAX_LETTRE = 10, // Taille pour la saisi d'une lettre ( si jamais
y'a des saisis de plusieurs lettres )
    NB_CARACTERES_STOCKE = 3 // Cas échange un chevalet avec la pioche
( une lettre ). Pour extraire la lettre à la 3ème position.
};

/**
 * @brief Ajoute les éléments de deux vecteurs dans un vecteur. ( Rail )
 * @param[in,out] v L'adresse d'un vecteur. ( Rail )
 * @param[in] m1 L'adresse d'un vecteur.
 * @param[in] m2 L'adresse d'un vecteur.

```

```

*/
void ajoutRail(Vecteur* r, const Vecteur* m1, const Vecteur* m2);

/**
 * @brief Met en place les éléments d'un vecteur ( Rail ) à partir de deux
vecteurs.
 * Les deux vecteurs sont les mots formés par chacunes des joueurs.
 * @param[in,out] r L'adresse d'un vecteur. ( Rail )
 * @param[in] m1 L'adresse d'un vecteur. ( Mot formé par joueur 1 )
 * @param[in] m2 L'adresse d'un vecteur. ( Mot formé par joueur 2 )
 */
void MisEnPlaceRail(Vecteur* r, const Vecteur* m1, const Vecteur* m2);

/**
 * @brief Affiche les éléments dans un vecteur.
 * Les éléments contenus dans rail.
 * @param[in,out] r L'adresse d'un vecteur.
 */
void afficheRail(Vecteur* r);

/**
 * @brief Copie les éléments d'un vecteur dans un autre vecteur.
 * @param[out] v1 L'adresse du vecteur cible.
 * @param[in] v2 L'adresse du vecteur source.
 */
void copieRail(Vecteur* v1, Vecteur* v2);

/**
 * @brief Inverse les éléments d'un vecteur.
 * Ce au début se trouve à la fin et inversement.
 * @param[in,out] r L'adresse du vecteur.
 */
void swap(Vecteur* r);

/**
 * @brief Vérification d'élément présents dans le vecteur.
 * @param[in,out] r L'adresse d'un vecteur.
 * @param[in] mot Chaîne de caractère à rechercher.
 * @return 1 si le mot est trouvée et 0 en cas d'échec.
 */
int DansRail(const Vecteur* r, const ItemV* mot);

/**
 * @brief Contenu de l'élément dans la parenthèse
 * @param[in] saisi Une chaîne de caractères lors de la saisi.
 * @param[out] r L'élément contenant uniquement dans la parenthèse.
 */
void dansParentheses(const ItemV* saisi, ItemV* r);

/**
 * @brief Lit le contenu sans la parenthèse.
 * @param[in] saisi Une chaîne de caractères lors de la saisi.
 * @param[out] mot L'élément sans les parenthèses.
 * @param[in] debut La position initiale pour extraire le mot.
 */
void lireMotSansParentheses(const ItemV* saisi, ItemV* mot, int debut);

/**
 * @brief Contenu de l'élément en dehors des parenthèses.
 * @param[in] mot Chaîne de caractères
 * @param[out] c L'élément hors parenthèses.

```

```

* @param[int] debut La position initiale pour extraire l'élément.
*/
void HorsParentheses(const ItemV* mot, ItemV* c, int debut);

/**
* @brief Changement apportés au vecteur.
* Modification du Rail
* @param[in,out] r L'adresse d'un vecteur.
* @param[in] mot Chaîne de caractère
* @param[in,out] j L'adresse d'un vecteur.
* @return 1 modification apporté et 0 en cas d'échec.
*/
int editRail(Vecteur* r, const ItemV* mot, Vecteur* j);

/**
* @brief Extraire un élément d'un vecteur. ( Rail )
* @param[in,out] r L'adresse d'un vecteur.
* @param[out] c L'élément extrait.
* @param[in] x La position initiale pour extraire l'élément.
*/
void extraireRail(Vecteur* r, ItemV* c, int x);

/**
* @brief Ajout d'un ou des élément(s) à la fin d'un vecteur. ( Rail )
* @param[in,out] r L'adresse d'un vecteur.
* @param[in] m L'élément à ajouter.
*/
void ajoutArr(Vecteur* r, const ItemV* m);

/**
* @brief Ajout d'un ou des élément(s) au début d'un vecteur. ( Rail )
* @param[in,out] r L'adresse d'un vecteur.
* @param[in] m L'élément à ajouter.
*/
void ajoutAvant(Vecteur* r, const ItemV* m);

/**
* @brief Ajustement d'un ou des élément(s) au début d'un vecteur.
* Opération rail lié à celle du joueur. Ajout de l'élément enlever du rail
au joueur.
* @param[in,out] r L'adresse d'un vecteur.
* @param[in,out] j L'adresse d'un vecteur.
*/
void ajustAvant(Vecteur* r, Vecteur* j);

/**
* @brief Ajustement d'un ou des élément(s) à la fin d'un vecteur.
* Opération rail lié à celle du joueur. Ajout de l'élément enlever du rail
au joueur.
* @param[in,out] r L'adresse d'un vecteur.
* @param[in,out] j L'adresse d'un vecteur.
* @param[in] c L'élément utile à la modification.
*/
void ajustD(Vecteur* r, Vecteur* j, ItemV c);

/**
* @brief Vérifie des éléments dans un vecteur.
* Si l'élément est dans les deux vecteurs. ( Rail et Joueur )
* @param[in,out] r L'adresse d'un vecteur.
* @param[in] mot L'élément à vérifier.
* @param[in] j L'adresse d'un vecteur.

```

```

* @return 1 si l'élément est trouvé et 0 en cas d'échec
*/
int VerifRail(Vecteur* r, const ItemV* mot);

/**
* @brief Met à jour les éléments d'un vecteur.
* On réinitialise et on ajoute les éléments dans ces vecteurs.
* @param[in,out] r1 L'adresse d'un vecteur.
* @param[in,out] r2 L'adresse d'un vecteur.
*/
void updRail(Vecteur* r1, Vecteur* r2);

```

Rail.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <assert.h>
#include "Rail.h"

/*Fonctions permettant de gérer et vérifier les chevalets sur le rail*/

void ajoutRail(Vecteur* r, const Vecteur* m1, const Vecteur* m2) {
    for (int i = 0; i < m1->nbElements; i++) {
        if (!ajouter(r, m1->elements[i])) {
            printf("Erreur d'ajout 1er mot dans Rail\n");
            break;
        }
    }
    for (int i = 0; i < m2->nbElements; i++) {
        if (!ajouter(r, m2->elements[i])) {
            printf("Erreur d'ajout 2eme mot dans Rail\n");
            break;
        }
    }
}

void MisEnPlaceRail(Vecteur* r, const Vecteur* m1, const Vecteur* m2) {
    int result = strcmp(m1->elements, m2->elements);
    if (result < 0) {
        ajoutRail(r, m1, m2);
    }
    else if (result > 0) {
        ajoutRail(r, m2, m1);
    }
    else {
        ajoutRail(r, m1, m2);
    }
}

void afficheRail(Vecteur* r) {
    for (int i = 0; i < r->nbElements; i++) {
        printf("%c", r->elements[i]);
    }
    printf("\n");
}

void copieRail(Vecteur* v1, Vecteur* v2) {
    for (int i = 0; i < v2->nbElements; i++) {

```



```

        if (!ajouter(v1, v2->elements[i])) {
            printf("Erreur d'ajout\n");
            break;
        }
    }
}

void swap(Vecteur* r) {
    assert(r != NULL);
    for (int i = 0; i < r->nbElements / 2; i++) {
        ItemV temp = r->elements[i];
        r->elements[i] = r->elements[r->nbElements - i - 1];
        r->elements[r->nbElements - i - 1] = temp;
    }
}

int DansRail(const Vecteur* r, const ItemV* mot) {
    int i = 0;
    int m = 0;

    if (strncmp(r->elements, mot, strlen(mot)) == 0) {
        return 1;
    }
    else if (strncmp(r->elements + MAX_R - strlen(mot), mot, strlen(mot))
== 0) {
        return 1;
    }
    return 0; // Mot pas au début ou vers la fin du rail
}

void dansParentheses(const ItemV* saisi, ItemV* r) {
    int idx = 0;
    int dansParentheses = 0;

    for (int i = 0; saisi[i] != '\0'; i++) { //copie à partir du 3ème.
        if (saisi[i] == '(') {
            dansParentheses = 1;
        }
        else if (saisi[i] == ')') {
            dansParentheses = 0;
        }
        else {
            if (dansParentheses) {
                r[idx++] = saisi[i];
            }
        }
    }
    r[idx] = '\0';
}

void lireMotSansParentheses(const ItemV* saisi, ItemV* mot, int debut) {
    int idx = 0; //Indice remplissage

    for (int i = debut; saisi[i] != '\0'; i++) {
        if (saisi[i] != '(' && saisi[i] != ')') {
            mot[idx++] = saisi[i];
        }
    }

    mot[idx] = '\0'; //termine chaîne
}

```

```

void HorsParentheses(const ItemV* mot, ItemV* c, int debut) {
    int dansP = 0;
    int idx = 0;

    for (int i = debut; mot[i] != '\0'; i++) {
        if (mot[i] == '(') {
            dansP = 1;
        }
        else if (mot[i] == ')') {
            dansP = 0;
        }
        else if (!dansP) {
            c[idx++] = mot[i];
        }
    }
    c[idx] = '\0';
}

int editRail(Vecteur* r, const ItemV* mot, Vecteur* j) {
    int d = 0;
    int f = 0;
    ItemV horsP[MAX_SAISI] = "";
    ItemV dansP[MAX_SAISI] = "";
    ItemV sansP[MAX_SAISI] = "";
    ItemV reste[MAX_SAISI] = "";
    dansParentheses(mot, dansP);
    HorsParentheses(mot, horsP, DEBUT_MOT);
    lireMotSansParentheses(mot, sansP, DEBUT_MOT);
    int taille = strlen(horsP);

    while (1) {
        if (strncmp(r->elements, dansP, strlen(dansP)) == 0) { //si dansP
trouvé au début rail
            if (strncmp(sansP, horsP, strlen(horsP)) == 0) { //si horsP
trouvé au début mot
                d = 1;
                break;
            }
            else {
                return 0;
            }
        }
        else if (strncmp(r->elements + MAX_R - strlen(dansP), dansP,
strlen(dansP)) == 0) { //fin Rail
            if (strncmp(sansP + strlen(sansP) - strlen(horsP), horsP,
strlen(horsP)) == 0) { //si horsP trouvé au début
                f = 1;
                break;
            }
            else {
                return 0;
            }
        }
        else {
            return 0;
        }
    }

    if (d == 1) {

```

```

    ajoutAvant(r, horsP);
    extraireRail(r, &reste, taille);
    for (int x = 0; x < strlen(horsP); x++) {
        ajustD(r, j, reste[x]); //probleme ici je pense
        //suppDerniereL(r);
        //supp lettre situé à la fin de rail
    } //rail ou lettre dans rail non supp
    return 1;
}

else if (f == 1) {
    ajoutArr(r, horsP); //Ajouter dans Rail

    for (int k = 0; k < strlen(horsP); ++k) {
        ajustAvant(r, j);
    }
    return 1;
}
else {
    printf("erreur");
    return 0;
}

return 0;
}

void extraireRail(Vecteur* r, ItemV* c, int x) {
    int idx = 0; //Indice remplissage
    for (int i = MAX_R; r->elements[i] != '\0'; i++) {
        c[idx++] = r->elements[i];
    }
    c[idx] = '\0'; //termine chaîne
}

void ajoutArr(Vecteur* r, const ItemV* m) {
    size_t tailleM = strlen(m);

    if (r->nbElements + tailleM >= r->capacite) {
        r->capacite = (r->nbElements + tailleM);
        r->elements = realloc(r->elements, r->capacite * sizeof(ItemV));
        if (r->elements == NULL) {
            printf("Erreur : allocation (ajoutArr) \n");
            exit(EXIT_FAILURE);
        }
    }
    ajoutMot(r, m);
}

void ajoutAvant(Vecteur* r, const ItemV* m) {
    size_t tailleM = strlen(m);

    if (r->nbElements + tailleM >= r->capacite) {
        r->capacite = r->nbElements + tailleM; // Ajuste la capacité avec
une marge (+1)
        r->elements = realloc(r->elements, r->capacite * sizeof(ItemV));
        if (r->elements == NULL) {
            printf("Erreur : allocation (ajoutAvant)\n");
            exit(EXIT_FAILURE);
        }
    }
}

```

```

    memmove(r->elements + tailleM, r->elements, r->nbElements + 1);
//Inclure '\0'
    memcpy(r->elements, m, tailleM);

    r->nbElements += tailleM;

}

void ajustAvant(Vecteur* r, Vecteur* j) {
    int FACTEUR = 2;
    if (r->nbElements > 0) {

        if (j->nbElements == j->capacite) {
            // Réallocation pour agrandir j->elements
            j->capacite = (j->capacite == 0) ? 1 : j->capacite * FACTEUR;
// Double la capacité (si la capacité était 0, initialisez-la à 1)
            j->elements = (ItemV*)realloc(j->elements, sizeof(ItemV) *
j->capacite);
            if (j->elements == NULL) {
                printf("Erreur (ajustAvant) \n");
                exit(EXIT_FAILURE);
            }
        }

        // Ajoute le premier élément du rail au joueur
        j->elements[j->nbElements] = r->elements[0];
        j->nbElements++;

        // Décale les éléments restants dans le rail
        memmove(r->elements, r->elements + 1, r->nbElements - 1);
        r->nbElements--;

        // Réallocation pour r->elements
        r->elements = (ItemV*)realloc(r->elements, sizeof(ItemV) *
r->nbElements);
        if (r->nbElements > 0 && r->elements == NULL) {
            printf("Erreur : allocation mémoire échouée\n");
            exit(EXIT_FAILURE);
        }
    }
    else {
        printf("Erreur : rail vide.\n");
    }
}

void ajustD(Vecteur* r, Vecteur* j, ItemV c) {
    int FACTEUR = 2;
    if (r->nbElements > 0 && c > 0) {

        if (j->nbElements == j->capacite) {
            // Réallocation pour agrandir j->elements
            j->capacite = (j->capacite == 0) ? 1 : j->capacite * FACTEUR;
// Double la capacité (si la capacité était 0, initialisez-la à 1)
            j->elements = (ItemV*)realloc(j->elements, sizeof(ItemV) *
j->capacite);
            if (j->elements == NULL) {
                printf("Erreur (ajustAvant) \n");
                exit(EXIT_FAILURE);
            }
        }
    }
}

```

```

    }

    // Ajoute l'élément à la fin de j->elements
    j->elements[j->nbElements] = c;
    j->nbElements++;

    // Supprime le dernier élément du rail (r)
    r->nbElements--;
    r->elements = (ItemV*)realloc(r->elements, sizeof(ItemV) *
r->nbElements);
    if (r->nbElements > 0 && r->elements == NULL) {
        printf("Erreur : allocation mémoire échouée\n");
    }
}
else {
    printf("Pas assez de lettres dans le rail ou caractère
invalide.\n");
}
}

int VerifRail(Vecteur* r, const ItemV* mot) { //enlever vecteur j
    int d = 0;
    int f = 0;
    ItemV horsP[MAX_SAISI] = "";
    ItemV dansP[MAX_SAISI] = "";
    ItemV sansP[MAX_SAISI] = "";
    ItemV reste[MAX_SAISI] = "";
    dansParentheses(mot, dansP);
    HorsParentheses(mot, horsP, DEBUT_MOT);
    lireMotSansParentheses(mot, sansP, DEBUT_MOT);
    int taille = strlen(horsP);

    while (1) {
        if (strncmp(r->elements, dansP, strlen(dansP)) == 0) { //si dansP
trouvé au début rail
            if (strncmp(sansP, horsP, strlen(horsP)) == 0) { //si horsP
trouvé au début mot
                d = 1;
                return 1;
            }
            else {
                return 0;
            }
        }
        else if (strncmp(r->elements + MAX_R - strlen(dansP), dansP,
strlen(dansP)) == 0) { //fin Rail
            if (strncmp(sansP + strlen(sansP) - strlen(horsP), horsP,
strlen(horsP)) == 0) { //si horsP trouvé au début
                f = 1;
                return 1;
            }
            else {
                return 0;
            }
        }
        else {
            return 0;
        }
    }
}
}

```

```

void updRail(Vecteur* r1, Vecteur* r2) {
    detruireVecteur(r1);
    initVecteur(r1, MAX_R);
    copieRail(r1, r2);
    swap(r1);
}

```

File.h

```

#pragma once

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "Vecteur.h"

/**
 * @file File.h
 * @brief Composant de file à capacité fixée.
 */
enum {
    LETTRES = 21, // Nombre de lettres différentes dans l'alphabet donné
    MAX_CHEVALETS = 88 // Nombre totale de chevauxets dans une partie
};

typedef Vecteur File; // File même structure que Vecteur (typedef struct)
typedef ItemV ItemF; // pour la cohérence des noms

/**
 * @brief Initialisation d'une file comme étant vide.
 * Après son utilisation, la mémoire occupée par une file doit être libérée
 * en invoquant la fonction @ref detruireFile.
 * @param[in,out] f L'adresse du File.
 * @param[in] capacite La capacité initiale du vecteur.
 */
void initFile(File* f, int capacite);

/**
 * @brief Vérifie file est vide.
 * @param[in] f L'adresse de la file.
 * @return 1 si la file est vide, sinon 0
 */
int estVide(File f);

/**
 * @brief Retourne l'élément d'une file se trouvant au début.
 * @param[in] f L'adresse de la file.
 * @return Le prochain élément devant sortir d'une file ( non vide )
 */
ItemF premier(const File* f);

/**
 * @brief Ajoute un élément dans une file. Celui si sortira de la file
 * après tous les éléments déjà présents.
 * @param[in,out] f L'adresse de la file.
 * @param[in] it L'élément à ajouter.
 */
void entrer(File* f, ItemF it);

```

```

/**
 * @brief Fait sortir un élément d'une file ( non vide ) et retourne sa valeur.
 * @param[in] f L'adresse de la file.
 * @return L'élément à retiré.
 */
ItemF sortir(File* f);

/**
 * @brief Libère l'espace mémoire occupé par une file.
 * @param[in,out] f L'adresse de la file.
 */
void detruireFile(File* f);

/**
 * @brief Affiche les éléments de la file. ( Pioche )
 * @param[in,out] f L'adresse de la file.
 */
void afficherPioche(File f);

/**
 * @brief Création de la pioche.
 * Cette fonction remplit la file avec des lettres
 * selon les quantités prédéfinies pour chaque lettre.
 * @param[in,out] f L'adresse de la file.
 * @pre La file f doit être vide
 */
void creerDeck(File* f);

/**
 * @brief Mélange les éléments de la file de manière aléatoire.
 * @param[in,out] f L'adresse de la file.
 */
void melange(File* f);

/**
 * @brief Ajoute les 12 premiers éléments de la file à un vecteur.
 * @param v [in, out] Pointeur vers le vecteur.
 * La fonction modifie le vecteur en y ajoutant des éléments
 depuis la file.
 * @param f [in, out] Pointeur vers la file.
 * La fonction modifie la file en retirant des éléments pour
 les ajouter au vecteur.
 */
void ajout(Vecteur* v, File* f);

/**
 * @brief retire un élément de la file et l'ajoute au vecteur.
 * @param[in,out] v L'adresse d'un vecteur. La fonction modifie le vecteur en y
 ajoutant un élément.
 * @param[in,out] f L'adresse d'une file. La fonction modifie la file en retirant
 un élément.
 */
void distribution(Vecteur* v, File* f);

/**
 * @brief Ajoute les éléments dans une file.
 * @param[in,out] f L'adresse d'une file. La fonction modifie la file en y
 ajoutant un élément.
 * @param[in] c L'élément à ajouter dans la file. L'élément 'c' est ajouté à la
 fin de la file.
 */
void ajoutPioche(File* f, ItemV c);

```

File.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <time.h>
#include "File.h"

/**
@file File.h
@brief Composant de file à capacité fixée.
*/

// Initialiser une file vide
void initFile(File* f, int capacite) {
    initVecteur(f, capacite); // Initialiser la file comme un vecteur
}

// Vérifier si la file est vide
int estVide(File f) {
    return f.nbElements == 0;
}

ItemF premier(const File* f) {
    return obtenir(f, 0);
}

void entrer(File* f, ItemF it) {
    ajouter(f, it);
}

ItemF sortir(File* f) {
    ItemF it = premier(f);
    supprimer(f, 0);
    return it;
}

void detruireFile(File* f) {
    detruireVecteur(f);
}

// Fonction pour afficher la file
void afficherPioche(File f) {
    for (int i = 0; i < f.nbElements; i++) {
        printf("%c", f.elements[i]); // Afficher chaque lettre dans la
file
    }
    printf("\n");
}

// Fonction pour créer le deck de chevalets
void creerDeck(File* f) {
    // Vérifier si la file est vide
    assert(estVide(*f));

    // Liste des lettres disponibles et leur quantité
    const char lettres[LETTRES] = "ABCDEFGHILMNOPQRSTUV";

```



```

    const unsigned int quantites[LETTRES] = {
        9, 1, 2, 3, 14, 1, 1, 1, 7, 1, 5, 3, 6, 5, 2, 1, 6, 7, 6, 5, 2
    };

    // Remplir la file avec les lettres
    for (unsigned int i = 0; i < LETTRES; i++) {
        for (unsigned int y = 0; y < quantites[i]; y++) {
            entrer(f, lettres[i]); // Ajouter chaque lettre en fonction de
sa quantité
        }
    }
}

void melange(File* f) {
    srand((unsigned int)time(NULL)); // Initialiser le générateur de
nombres aléatoires

    for (int i = f->nbElements - 1; i > 0; i--) {
        int j = rand() % (i + 1); // Générer un nombre aléatoire entre 0
et i

        // Échanger les lettres aux indices i et j
        char temp = f->elements[i];
        f->elements[i] = f->elements[j];
        f->elements[j] = temp;
    }
}

void ajout(Vecteur* v, File* f) {
    for (int i = 0; i < MAX_C; i++) {
        if (!ajouter(v, f->elements[i])) {
            printf("Erreur d'ajout\n");
            break;
        }
    }
}

void distribution(Vecteur* v, File* f) {
    if (f->nbElements > 0) {
        ajouter(v, sortir(f));
    }
    else {
        printf("Plus de lettres disponibles\n");
    }
}

void ajoutPioche(File* f, ItemV c) {
    for (int i = 0; i < 1; i++) {
        if (!ajouter(f, c)) {
            printf("Erreur d'ajout\n");
            break;
        }
    }
}

```

Vecteur.h

```
#pragma once
```

```

#include "Itemvecteur.h"

/**
 * @brief Conteneur stockant des éléments accessibles en
 * fonction de leur position (indice).
 */
enum {
    MAX_C = 12,      // Nombre de chevauxets d'un joueur au début du jeu
    MAX_R = 8,        // Taille du Rail égale à huit
    MOT_DE_QUATRE = 4, // Taille du mot pour la mise en place du jeu
    MAX_MOT = 8,      // Un mot de maximum 8 lettres qui peut être formés
};

typedef struct {
    int nbElements; // Nombre d'éléments présents dans le vecteur.
    int capacite;   // Nombre d'éléments maximal du vecteur.
    ItemV* elements; // Tableau (dynamique) de taille
<code>capacite</code>.
} Vecteur;

/**
 * @brief Initialise un vecteur d'une capacité donnée contenant aucun
 élément.
 * Après son utilisation, la mémoire occupée par un vecteur doit être
 libérée
 * en invoquant la fonction @ref detruireVecteur.
 * @param[out] v L'adresse du vecteur à initialiser.
 * @param[in] capacite La capacité initiale du vecteur.
 * @return 0 en cas d'échec (manque de mémoire disponible) et 1 en cas de
 succès.
 * @pre <code>capacite</code> doit être supérieur ou égal à 1.
 */
int initVecteur(Vecteur* v, int capacite);

/**
 * @brief Retourne le nombre d'éléments présents dans un vecteur.
 * @param[in] v L'adresse du vecteur.
 * @return Le nombre d'éléments contenu dans <code>v</code>.
 */
int taille(const Vecteur* v);

/**
 * @brief Ajoute un élément dans un vecteur. Cet élément est ajouté après
 ceux déjà présents.
 * @param[in,out] v L'adresse du vecteur.
 * @param[in] it L'élément devant être ajouté.
 * @return 0 en cas d'échec (manque de mémoire disponible) et 1 en cas de
 succès.
 */
int ajouter(Vecteur* v, ItemV it);

/**
 * @brief Retourne l'élément d'un vecteur se trouvant à une position
 donnée.
 * @param[in] v L'adresse du vecteur.
 * @param[in] i La position (i.e. l'indice).
 * @return L'élément de <code>v</code> se trouvant à l'indice
<code>i</code>.
 * @pre La valeur de <code>i</code> doit être comprise entre 0 et
 * <code>(taille(v) - 1)</code> (inclus).
 */

```

```

ItemV obtenir(const Vecteur* v, int i);

/**
 * @brief Supprime un élément d'un vecteur.
 * @param[in,out] v L'adresse du vecteur.
 * @param[in] i La position (i.e. l'indice) de l'élément devant être
supprimé.
 * @pre La valeur de <code>i</code> doit être comprise entre 0 et
 * <code>(taille(v) - 1)</code> (inclus).
 */
void supprimer(Vecteur* v, int i);

/**
 * @brief Libère l'espace mémoire occupé par un vecteur. Après avoir été
détruit, il ne doit
 * pas être ré-employé sans avoir été ré-initialisé. Toute autre opération
peut donner des
 * résultats incohérent ou même provoquer l'arrêt brutal du programme.
 * @param[in,out] v L'adresse du vecteur.
 */
void detruireVecteur(Vecteur* v);

```

Vecteur.c

```

#include <assert.h>
#include <stdlib.h>
#include "Vecteur.h"

/**
 * @brief Conteneur stockant des éléments accessibles en
 * fonction de leur position (indice).
 */

int initVecteur(Vecteur* v, int capacite) {
    assert(capacite > 0);
    v->capacite = capacite;
    v->nbElements = 0;
    v->elements = (ItemV*)malloc(sizeof(ItemV) * capacite);
    return v->elements != NULL;
}

int taille(const Vecteur* v) {
    return v->nbElements;
}

int ajouter(Vecteur* v, ItemV it) {
    const int FACTEUR = 2;
    if (v->nbElements == v->capacite) {
        ItemV* tab = (ItemV*)realloc(v->elements, sizeof(ItemV) *
v->capacite * FACTEUR);
        if (tab == NULL)
            return 0;
        v->capacite *= FACTEUR;
        v->elements = tab;
    }
    v->elements[v->nbElements++] = it;
    return 1;
}

```

```

ItemV obtenir(const Vecteur* v, int i) {
    assert(i >= 0 && i < v->nbElements);
    return v->elements[i];
}

void supprimer(Vecteur* v, int i) {
    assert(i >= 0 && i < v->nbElements);
    for (++i; i < v->nbElements; ++i)
        v->elements[i - 1] = v->elements[i];
    --v->nbElements;
}

void detruireVecteur(Vecteur* v) {
    free(v->elements);
}

```

Itemvecteur.h

```

#pragma once

/** @brief Définition du type ItemV pour représenter des éléments de type
caractère.
*/
typedef char ItemV;

```